影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 16 章 物件偵測 YOLO 實戰

本章學習目標

- 區分影像分類、物件偵測、語意分割與實例分割四種視覺任務。
- 說明兩階段與單階段偵測器的差異，理解 YOLO「只看一次」的核心思想。
- 運用 IoU、NMS、mAP 等關鍵概念評估與後處理物件偵測結果。
- 以 Ultralytics YOLO26 完成推論、訓練自訂資料集與模型驗證，並理解免 NMS 端到端推論的意義。
- 操作 YOLO 的分割、姿態估計與追蹤等延伸功能，並串接第 12 章視訊處理。
- 思辨電腦視覺技術的倫理責任，理解工程師在技術應用中的角色。

叫獸開講|YOLO 之父的告別：當創造者選擇放下

2016 年，華盛頓大學的研究生 Joseph Redmon 在頂級會議 CVPR 上發表了一篇論文，提出一個名叫 YOLO(You Only Look Once，「你只需看一次」)的物件偵測演算法。在那之前，主流的偵測方法要分兩階段：先產生大量候選框、再逐一分類，又慢又笨重。Redmon 的想法極為大膽：何不讓神經網路「看一次」整張影像，就直接同時輸出所有物件的位置與類別?這個一氣呵成的設計讓偵測速度快到能即時執行，論文一舉拿下 OpenCV 群眾票選獎，隔年的改良版 YOLO9000 更獲得 CVPR 最佳論文榮譽提名。YOLO 成了物件偵測的代名詞，從自駕車、工廠檢測到手機相機，無所不在。

Redmon 本人也是個充滿個人風格的研究者——他的個人網站與履歷以幽默不羈聞名，論文裡甚至夾雜著玩笑與自嘲。但在 2018 年的 YOLOv3 論文結尾，他寫下了一段不太像技術論文的文字，流露出對自己研究可能被誤用的不安。

2020 年 2 月，這份不安化為行動。當時學術界正在討論 NeurIPS 會議的新規定——要求論文作者加寫一段「廣泛影響」，說明研究可能帶來的正面與負面社會後果。有人質疑：研究者真的有必要因為「廣泛影響」而決定不投稿嗎?就在這場討論中，Redmon 現身分享了自己的親身經歷：他說，他已經停止電腦視覺研究了，因為他看見自己的研究成果帶來的影響——儘管熱愛這份工作，但軍事應用與隱私疑慮已經到了無法忽視的地步。這位親手創造出當代最流行偵測演算法的人，選擇了放下。

這件事在社群中引發了持久的辯論，而雙方都有道理。反對者說：技術本身沒有道德，刀子可以殺人也可以救人，問題出在使用者而非發明者;而且你不做，別人也會做，退出只是讓自己失去影響力。支持者則說：創造者對自己創造物的流向並非全然無責，當一項技術的下行風險遠大於上行效益時，選擇不推進它，本身就是一種負責任的表態。

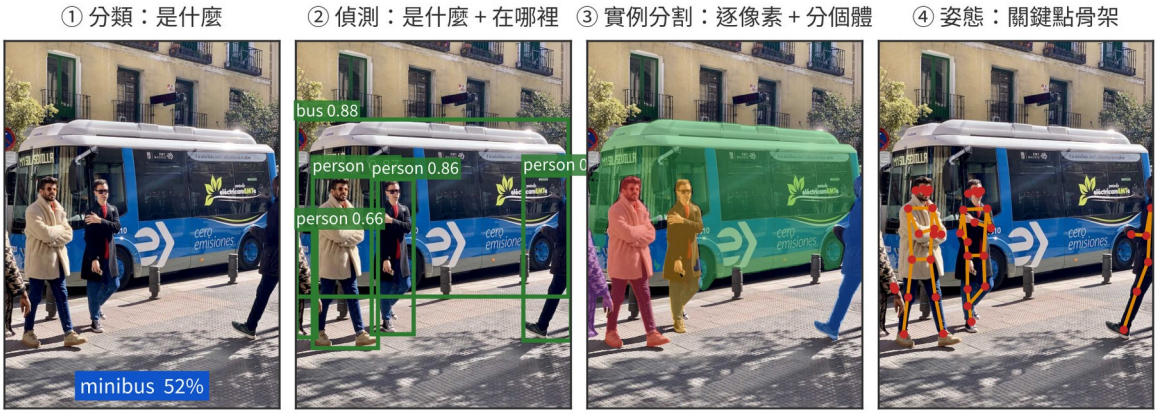
這一章我們要學的，正是 Redmon 創造的 YOLO——它強大、快速、易用，是您在課程中要親手部署的主力工具。但在按下訓練指令之前，叫獸希望你先記住這個故事。作為工程師，我們手上的技術是中性的，但我們選擇把它用在哪裡、為誰服務、防範什麼樣的濫用，從來就不是中性的。這門課從第 7 章的「還原 vs. 腦補」、第 9 章的醫學影像、第 12 章的照護監控、第 14 章的資料偏誤，一路鋪陳到這裡，談的都是同一件事：技術能力愈強大，對使用者處境的體貼與對後果的想像，就愈是專業素養不可分割的一部分。

思考題：「技術本身沒有道德，問題在使用者」與「創造者對技術的流向也有責任」——你比較認同哪一方?如果你設計的視覺系統可能被用於你不認同的用途，你會怎麼做?(想想：授權條款、使用限制、公開透明、或如 Redmon 一般選擇退出)

16.1 四種視覺任務

進入 YOLO 之前，必須先分清楚四種常被混淆的視覺任務。它們的差別在於「輸出的精細程度」，難度也依序遞增：

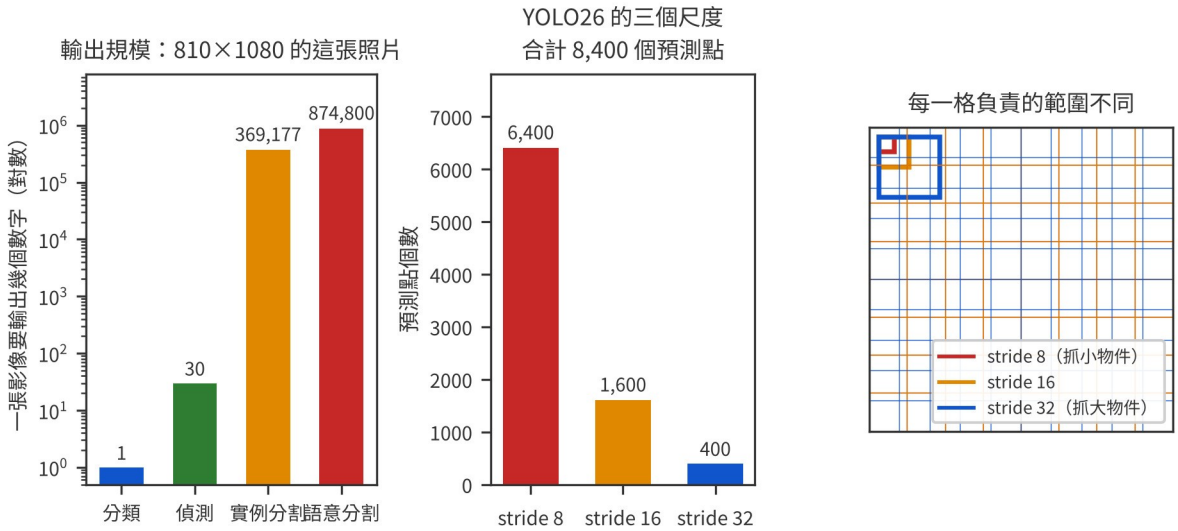
1. 影像分類(Classification)：輸出整張影像屬於哪一類——「這是一張貓的照片」。這是第 14、15 章的主題，只回答「是什麼」，不回答「在哪裡」。
2. 物件偵測(Object Detection)：輸出影像中每個物件的類別與位置(邊界框)——「這裡有一隻貓、那裡有兩個人」。它同時回答「是什麼」與「在哪裡」，是本章的主軸。
3. 語意分割(Semantic Segmentation)：為每一個像素標上類別——「這些像素是路面、那些是天空」。它給出精確的輪廓，但不區分同類的不同個體(三個人會被標成同一片「人」的區域)。這是第 9.6 節介紹過的 U-Net 所擅長的。
4. 實例分割(Instance Segmentation)：結合偵測與分割——不僅標出每個像素的類別，還區分同類的不同個體(三個人各自有獨立的輪廓與編號)。這是最精細、也最困難的任務。



16.1 節說四種任務的差別在「輸出的精細程度」。這張圖把同一張 810×1080 的照片餵給四個真實的 YOLO26 模型，讓差別自己顯現。①分類只回答「是什麼」：整張圖得到一個標籤（minibus，信心 52%），它連公車在畫面左邊還是右邊都不知道。②偵測同時回答「在哪裡」：5 個框，每個框是「類別 + 信心 + 四個座標」共 6 個數字，總共 30 個數字。③實例分割把框換成逐像素的遮罩：5 張遮罩、合計 369,177 個前景像素，所以它能算出物件的真實面積，而框只能給外接矩形（圖 16-18 會量這個差）。④姿態估計輸出的是結構：每個人 17 個關鍵點、各帶 x、y 與信心，5 個人共 255 個數字，於是「手舉起來了沒」這種問題才問得出口。注意這四件事用的是同一套骨幹網路、同一份 COCO 訓練資料，差別只在最後的預測頭與標註形式——換一個權重檔就換一種任務，這正是 16.5 節「整合的視覺工具箱」的意思。另外請記住語意分割沒有出現在這張圖上：它逐像素標類別但不分個體，三個人會融成一塊「人」；③之所以叫實例分割，就是因為每個人各有獨立的遮罩與編號。

圖 16-1　四種視覺任務：同一張照片，四種輸出

另有一個常見的延伸任務：姿態估計(Pose Estimation)，它偵測的不是物件的框，而是人體(或物體)的關鍵點——頭、肩、肘、腕、髖、膝、踝等骨架節點。它在動作分析、人機協作、以及第 12 章的跌倒偵測中極為重要。現代的 YOLO 已經把偵測、分割、姿態估計、追蹤整合進同一個工具中，這正是它成為業界主力的原因。



左圖把四種任務的輸出規模畫在對數軸上，一眼就看出 16.1 節那句「難度依序差各一」的物理來源：分類只要吐 1 個數字，偵測要吐 5×6 = 30 個，實例分割要為 369,177 個前景像素各做一次判斷，語意分割更是整張 810×1080 = 874,800 個像素都要。輸出規模差了五個數量級，標註成本也差不多是這個比例——畫一個框幾秒鐘，描一個輪廓要好幾分鐘，這就是為什麼實務上能用框解決的事，很少有人去做分割。中圖與右圖則回答「YOLO 一次看完全圖，那它到底預測了幾次」。把 YOLO26n 的偵測頭直接攤開來看，640×640 的輸入會輸出一個 84×8,400 的張量：84 是每個預測點的內容（4 個框座標 + 80 個 COCO 類別分數），8,400 則是 6,400 + 1,600 + 400——三個解析度的特徵圖，格子邊長分別是 8、16、32 個像素。這就是 16.2.3 節說 v3 引入「多尺度預測」的實體：細的格子多、負責小東西，粗的格子大、負責大東西，而這正是第 11 章影像金字塔的同一個思想（下一張圖會量它救回多少）。所以「只看一次」不是只預測一次，而是 8,400 個預測在同一次前向傳播裡一起算完——真正省下來的是重複的特徵萃取。

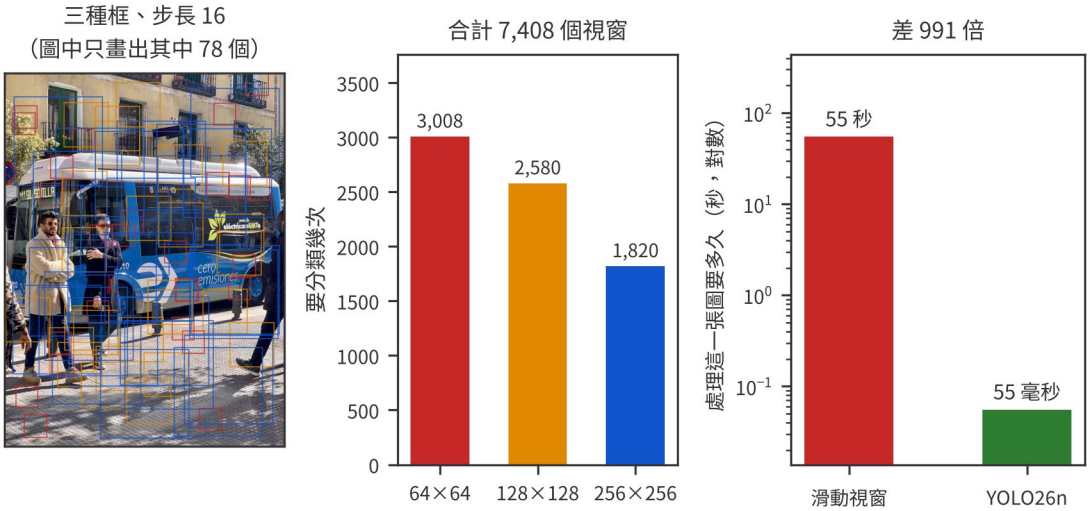
圖 16-2　輸出的資訊量，與 YOLO 的 8400 個預測點

16.2 物件偵測的演進與 YOLO 的核心思想

16.2.1 從傳統方法到深度學習

回顧本書，我們其實已經走過了物件偵測的整部演進史。第 10 章的 Viola-Jones(2001)以 Haar 特徵加級聯分類器，是第一代能即時執行的偵測器，但只擅長正面人臉;第 13 章的特徵匹配加 RANSAC，能定位特定物件但難以泛化到「所有的貓」；而傳統做法的共同侷限是：依賴人工設計特徵(第 15 章已詳述這個瓶頸)。

深度學習時代的偵測器分兩大流派。兩階段偵測器(如 R-CNN 系列)先產生候選區域、再對每個區域分類與微調框，準確度高但速度慢——因為同一張影像要被網路處理很多次。單階段偵測器(如 YOLO、SSD)則一次前向傳播就同時輸出所有物件的位置與類別，速度快得多，這正是 YOLO 名稱「You Only Look Once」的由來。



16.2 節說單階段偵測器「一次前向傳播就輸出所有結果」，而兩階段與更早的滑動視窗要「同一張影像被網路處理很多次」——這裡的「很多次」換算成秒。做法是老派的滑動視窗：拿一個分類器，用 64×64、128×128、256×256 三種框、步長 16 個像素掃過這張 810×1080 的照片，一共要分類 3,008 + 2,580 + 1,820 = 7,408 次。每一次要多久?本圖實測 YOLO26n-cls 分類一塊 224×224 的小圖是 7.4 毫秒，乘起來就是 55 秒——一張圖要跑快一分鐘。同一台機器上，YOLO26n 看完整張圖只要 55 毫秒，快了約 991 倍。差距的來源不是網路比較小（兩者骨幹規模相近），而是滑動視窗把重疊區域的特徵重複算了上萬遍，YOLO 則整張圖只萃取一次特徵，再讓上千個預測點共用這份特徵。這就是「只看一次」真正的意思。要公平的話得補一句：真正的兩階段偵測器（Faster R-CNN 之後）早就不這樣做了——它們同樣只萃取一次特徵，只是多了一個「提候選區域」的步驟，因此慢的是倍數而不是數量級。這張圖量的是最樸素的做法，用來說明「重複計算」的代價有多可怕。

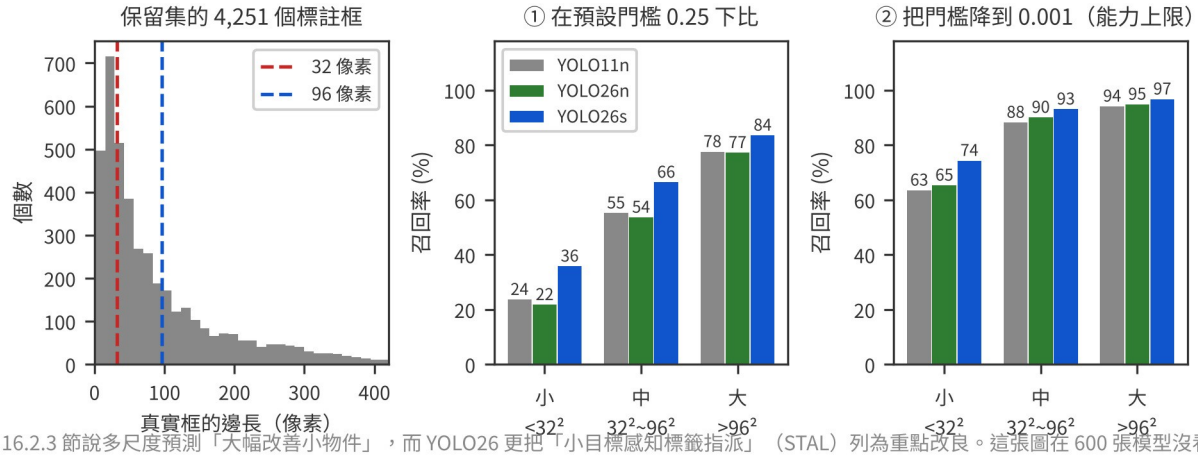
圖 16-3　滑動視窗的帳單：為什麼「只看一次」是關鍵

16.2.2 YOLO 的核心思想

YOLO 的巧思在於把「偵測」重新定義成一個「迴歸問題」：它把影像切成網格，讓網路對每個網格直接預測「這裡有沒有物件、框在哪裡、是什麼類別、信心多高」。整張影像只需通過網路一次，所有預測一氣呵成——這與兩階段方法「先提候選再逐一檢查」的思路截然不同，是一次漂亮的觀點翻轉(呼應第 9 章霍夫轉換的叫獸開講)。

這個設計還有一個額外好處：因為 YOLO 每次都看整張影像，它能利用全域的脈絡資訊(例如「這個區域周圍都是道路，所以這個模糊的物體比較可能是車而非船」)，

背景誤判反而較少。而它早期的弱點是對小物件與密集群體較不擅長——這在後續版本中已大幅改善，但必須說清楚「改善」不等於「解決」。圖 16-4 以 YOLO26n 在 600 張模型沒看過的 COCO 影像上實測(信心門檻 0.25)：大物件的召回率有 77%，中物件 54%，面積小於 32×32 的小物件只剩 22%。就算把信心門檻降到底、問它「究竟找不找得到」，小物件也只有 65%，而大物件已經 95%;換成大一號的 YOLO26s 也只把小物件的上限拉到 74%——換更大的模型救不了小物件。真正有效的是提高輸入解析度或把大圖切塊偵測(圖 16-12)，而這兩招又各有代價。請把這件事記在心裡：當你的應用要抓的是遠方的果實、小螺絲或路口外的行人，預設設定給你的召回率可能只有兩成。



16.2.3 節說多尺度預測「大幅改善小物件」，而 YOLO26 更把「小目標感知標籤指派」（STAL）列為重點改良。這張圖在 600 張模型沒看過的 COCO 影像、4,251 個標註框上量它，順便示範一個很容易踩到的比較陷阱。左圖先看資料：面積小於 32^2 的小物件有 1,331 個、中 1,487 個、大 1,433 個——小物件不是邊緣案例，是常態。中圖①是多數人會做的比較：固定信心門檻 0.25。結果 YOLO26n 的小物件召回率 22% 竟然比上一代的 YOLO11n（24%）還低，中、大物件也都略輸。難道新版反而退步了?右圖②揭穿了答案：把門檻降到 0.001、也就是問「它究竟找不找得到」，YOLO26n 在三個尺寸上全部反超——小 63% → 65%、中 88% → 90%、大 94% → 95%。差別在「信心的刻度」不是「能力」：實測 YOLO26n 命中框的信心中位數是 0.418，YOLO11n 是 0.457——新模型比較保守，同一個 0.25 砍掉了它更多正確的預測。這就是為什麼比較模型要看與門檻無關的 mAP（圖 16-7），而不是在某個固定門檻下比召回率;也是為什麼換模型之後，門檻必須重新調（圖 16-11）。至於小物件本身：即使放到能力上限，YOLO26s 的小物件召回也只有 74%，而在可用的門檻下整批漏掉的 2,059 個框中有 1,042 個是小物件。架構的改良方向是對的，但問題沒有被解決，只是變得比較可以忍受;真正有效的仍是提高輸入解析度（圖 16-12）或把大圖切塊偵測。

圖 16-4　小物件有多難，以及固定門檻比較的陷阱

16.2.3 YOLO 的世代演進

YOLO 從 2016 年至今經歷了快速的世代更迭：v1 到 v3 由 Redmon 本人主導(叫獸開講的主角),v3 引入多尺度預測(呼應第 11 章多解析度思想)大幅改善小物件偵測;此後由社群與不同團隊接棒，陸續推出 v4、v5、v6、v7、v8，再到 YOLO11，以及本章採用的 YOLO26。演進的主軸始終不變：更快、更準、更容易使用，並逐步把分割、姿態估計、追蹤等功能整合進同一套框架。但 YOLO26 這一代有一個特別值得注意的轉折：它第一次讓「不需要非極大值抑制」成為可選的標準做法，把一個從 v1 就跟著 YOLO 的後處理步驟變成可以拿掉的東西。下一小節專門說明這件事與它帶來的連鎖效應。

本章實作採用 Ultralytics 提供的 YOLO26。它把整套流程封裝得極為簡潔——載入模型、推論、訓練自訂資料集都只需幾行程式，同時提供 n(nano)、s、m、l、x 等不同大小的模型，讓使用者依算力需求選擇：nano 版可在第 17 章的邊緣裝置上即時執行，x 版則追求最高精度。這個「同一架構、多種尺寸」的設計，正呼應了第 15 章

MobileNet 所代表的「夠好且能部署」的務實路線。依官方在完整 COCO 驗證集上的數據，YOLO26 五種尺寸的 mAP@0.5:0.95 落在 40.9% 到 57.5% 之間;而本章所有實測數字則取自 COCO val2017 的 600 張隨機子集(刻意避開模型訓練用過的影像，理由見圖 16-18)，因此與官方數字不會完全一致，但兩者的相對關係一致。

16.2.4 YOLO26 帶來的四個改變

每一代 YOLO 都宣稱更快更準，但多半是同一個架構的調校。YOLO26 不太一樣——它動到了四個會改變你寫程式方式的地方。這一小節逐一說明，並指出各自在本章哪一張圖被量過。

其一，也是影響最大的一項：免 NMS 的端到端推論。從 v1 到 YOLO11，偵測器的輸出都是「很多個互相重疊的框」，必須再跑一次非極大值抑制(16.3.2 節)才能得到最終結果。YOLO26 同時訓練兩個預測頭：預設的「一對多」(one-to-many)頭維持舊行為，讓多個預測點各報一個框、再交給 NMS 去重;另一個「一對一」(one-to-one)頭則被訓練成每個物件只報一個框，因此完全不需要 NMS。使用方式只是在推論驗證或匯出時加上 nms=False:

```
from ultralytics import YOLO
model = YOLO("yolo26n.pt")

# 預設:一對多預測頭,後面接 NMS
r1 = model.predict("street.jpg")

# 端到端:一對一預測頭,不跑 NMS,輸出固定 300 個框
r2 = model.predict("street.jpg", nms=False)
m  = model.val(data="coco.yaml", nms=False)
model.export(format="onnx", nms=False)
```

兩個頭的差別值得細看。一對多頭的原始輸出是一個 (N, 類別數 + 4, 8400) 的張量——8400 就是 16.2.2 節那三個尺度的預測點總數(圖 16-2);一對一頭的輸出則是 (N, 300, 6)，也就是最多 300 個框、每個框六個數字(四個座標 + 信心 + 類別)。後者不但不必跑 NMS，輸出形狀還是固定的，這對只接受靜態形狀的推論引擎特別友善。準確度要付多少代價?圖 16-10 在 600 張保留影像上實測，兩者的 mAP 幾乎相同;而圖 16-9 更顯示在物件彼此嚴重重疊的場景中，免 NMS 的頭反而明顯較好——因為它不會像 NMS 那樣把「兩個真的疊在一起的物件」誤判為重複而刪掉。官方另外報告在特定伺服器 CPU 上匯出 ONNX 後可快上約四成，但那是特定硬體與格式下的數字，請在自己的平台上重測。

其二，移除分布焦點損失(DFL)。從 YOLOv8 以來，框的邊界是用一組離散的機率分布來回歸的，預測出來的不是「邊界在哪」而是「邊界落在各個位置的機率」，再取期望值得到座標。這個設計精度好，但它讓預測頭多了一段只有框架自己懂的解碼邏輯，匯出到別的推論引擎時經常要手工重寫。YOLO26 把它拿掉，改回直接回歸座標、且不限制回歸範圍。對讀者的實際意義是：匯出變單純了，而這與免 NMS 是同一個方向——讓模型愈接近「一張計算圖就跑得完」，第 17 章的邊緣部署就愈輕鬆。

其三，訓練方法的兩項改良。漸進式損失(ProgLoss)讓訓練過程逐步把重心移到推論時真正會用的那個頭上，避免兩個頭互相拉扯;小目標感知標籤指派(STAL)則調整

了「哪些預測點該為某個物件負責」的規則，提高小物件取得正樣本的機會。此外最佳化器改用 MuSGD，是 SGD 與 Muon 的混合體，靈感來自大型語言模型的訓練經驗這些都是訓練端的改動，使用者不會直接碰到，但 STAL 的效果可以量：圖 16-4 在保留集上比較兩代模型，把信心門檻降到底之後，YOLO26n 的小物件召回率確實高於 YOLO11n。同一張圖也示範了一個重要的陷阱——若在固定的信心門檻下比較，結論會反過來，因為新模型的信心刻度比較保守。換版本之後，門檻必須重新調。

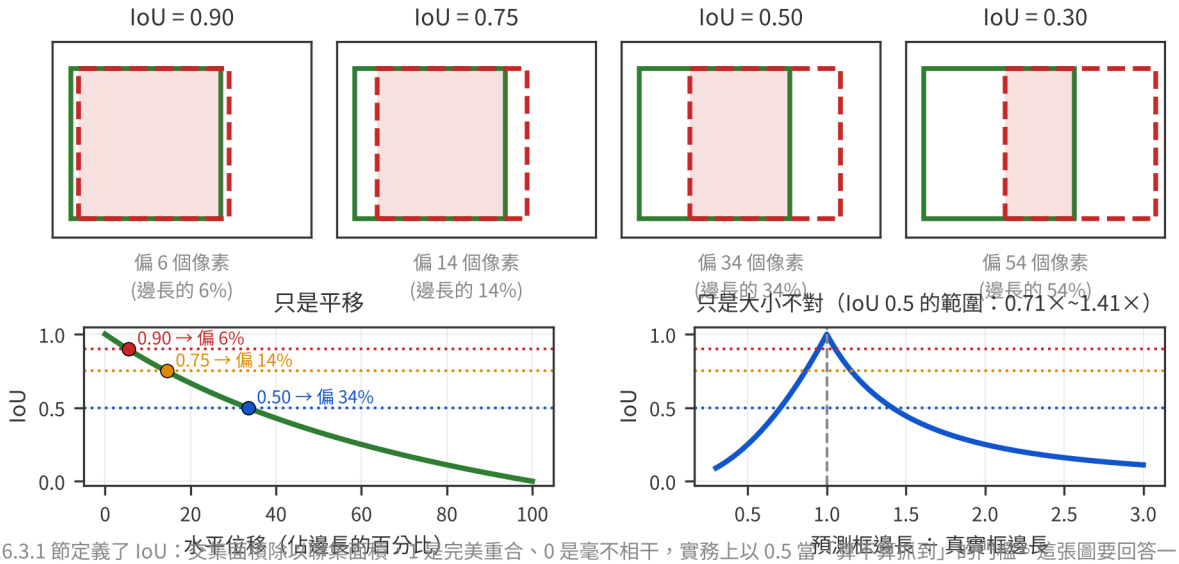
其四，任務家族再度擴大。除了原有的偵測、實例分割、分類、姿態估計與有向邊界框，YOLO26 另外提供語意分割與單眼深度估計的預訓練權重，權重檔名一律是 yolo26{尺寸}-{任務}.pt 的形式(例如 yolo26n-seg.pt、yolo26s-pose.pt、yolo26n-obb.pt)。16.5.4 節會逐一說明這些新任務各自回答什麼問題。

最後補一句使用上的提醒，它比上面任何一項都重要。本章其餘各節的觀念——IoU、NMS、mAP、信心門檻、標註品質、評估的誤差——沒有任何一項因為換版本而失效。免 NMS 拿掉的是一個後處理步驟，不是「框準不準」這個問題;STAL 改善的是小物件的召回率，不是「小物件很難」這個事實。新版本讓你少寫幾行程式、少調一個門檻，但判斷模型好不好的方法完全沒有改變。這也是為什麼本書把方法論放在工具之前。

16.3 物件偵測的關鍵概念

16.3.1 邊界框與 IoU

物件偵測的輸出是邊界框(bounding box)，通常以「左上角座標加寬高」或「中心座標加寬高」表示，並附帶類別與信心分數。要評估一個預測框準不準，需要一個量化的標準——這就是 IoU(Intersection over Union，交集聯集比)：把預測框與真實框的「重疊面積」除以「聯集面積」。IoU 為 1 代表完美重合，為 0 代表完全不重疊。實務上常以 IoU 大於某門檻(如 0.5)判定為「偵測正確」。這裡要補一句很少被講明的話：0.5 這個門檻其實相當寬鬆。對一個 100×100 的框，交集與聯集的算式給出 IoU = (100−d)/(100+d)，令它等於 0.5 得 d ≈ 33——也就是說預測框水平偏掉三分之一個身位，仍然算「正確偵測」;同理，框的邊長只要落在真實框的 0.71 到 1.41 倍之間，IoU 也還有 0.5(圖 16-5 的實測與這兩個算式完全吻合)。所以讀到「mAP@0.5 高達九成」時要意識到那是在問「有沒有找到東西」;要衡量「框畫得準不準」，必須看 mAP@0.5:0.95。若你的應用需要精確位置(機械手臂抓取、尺寸量測)，IoU 0.5 的評分根本不能用——它會讓一個偏了三分之一身位的框看起來一切正常。

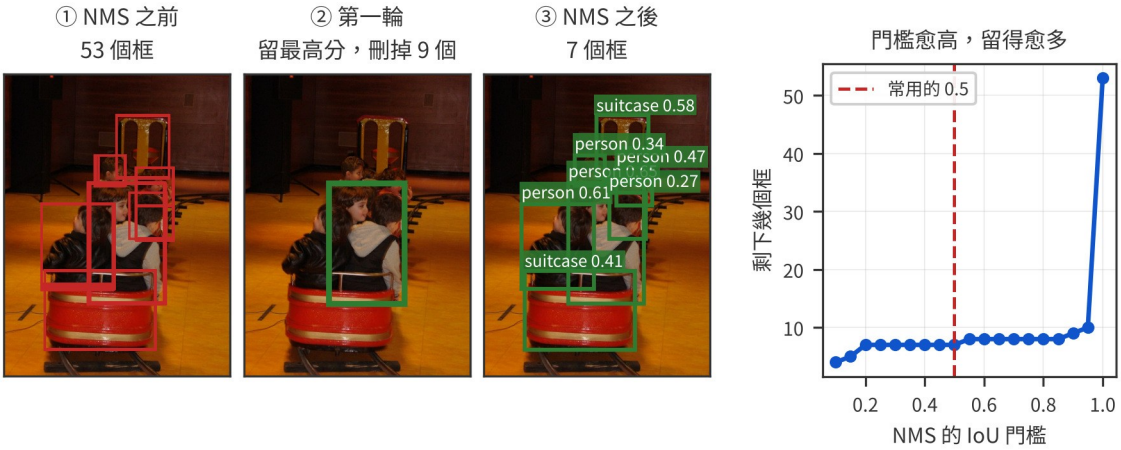


16.3.1 節定義了 IoU：交集面積除以聯集面積，1 是完美重合、0 是毫不相干，實務上以 0.5 當「算中算抓到」的門檻。這張圖要回答一個教科書通常不講的問題：0.5 到底有多鬆?上排是答案。對一個 100×100 的框，IoU 0.90 只容許水平偏 6 個像素（邊長的 6%），0.75 容許 14%，而 0.50 容許整整 34%——預測框偏掉三分之一個身位，仍然算「正確偵測」。右下圖量的是另一種錯法：框的位置對、大小不對。只要邊長落在真實框的 0.71 到 1.41 倍之間，IoU 就還有 0.5。也就是說，框只要大到 1.41 倍或縮到 0.71 倍，IoU 都還掛在及格線上。兩件事值得記下來。其一，當你讀到「mAP@0.5 達到 0.9」時，那是一個相當寬鬆的標準下的成績;要量「框畫得準不準」，就得看 mAP@0.5:0.95（圖 16-13）。其二，反過來說，如果你的應用真的需要精確的位置（機械手臂抓取、尺寸量測），那麼 IoU 0.5 的評分根本不能用——它會讓一個偏了三分之一身位的框看起來一切正常。指標要配合用途選，這是第 14.5 節那一課在偵測問題上的回聲。

圖 16-5　IoU 的幾何：偏一點點，分數掉多少

16.3.2 非極大值抑制(NMS)

偵測器對同一個物件，往往會輸出好幾個高度重疊的框。非極大值抑制(NMS)負責去蕪存菁：先取信心分數最高的框，再把與它 IoU 超過門檻的其他框都刪掉(視為重複)，接著對剩下的框重複這個程序，直到沒有框可處理。這個「保留局部最強、抑制鄰近重複」的思想，你應該覺得眼熟——它正是第 9 章 Canny 邊緣偵測第三步「非極大值抑制」的同名兄弟，兩者的精神完全一致。這種概念的跨章重現，是本書一再強調的：影像處理的核心思想並不多，但它們會以不同面貌反覆出現。不過 NMS 有一個無解的兩難必須先說：它分不清「同一個物件的重複框」與「兩個真的疊在一起的物件」。圖 16-9 在保留集裡找到的實例是兩個並排的同類物件——它們的真實框彼此 IoU 就有 0.66，只要 NMS 門檻設在這個值以下，其中一隻必定被當成重複而刪除，而且是在偵測器正確找到它之後才刪的。人群、貨架、果樹、停車場這些密集場景天天在發生這件事。實測把門檻從 0.2 調到 0.95，召回率上升而精確率大幅下降，所以這個門檻不是「調到最好」，而是「依場景調」。也正因為如此，YOLO26 的免 NMS 預測頭(16.2.4 節)才特別有價值：它根本不需要這個門檻，而且在物件重疊嚴重的場景中召回率上限比一對多頭高出約六個百分點(圖 16-9)。還要注意一個容易混淆的地方：NMS 的 IoU 門檻與評分用的 IoU 門檻是兩回事，前者問「這兩個預測是不是在講同一個物件」，後者問「這個預測跟標準答案夠不夠接近」;它們剛好同名、也剛好常常都設 0.5，但調錯了哪一個，後果完全不同。



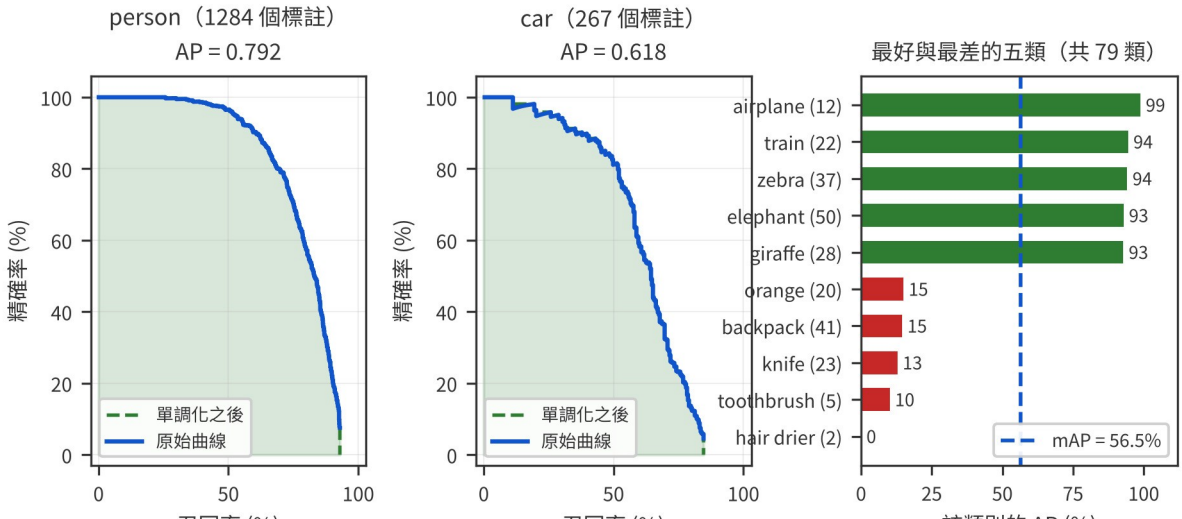
16.3.2 節描述了 NMS 的三個步驟：取信心最高的框、刪掉與它 IoU 超過門檻的其他框、對剩下的重複。這張圖把那三步畫在一張真實的 COCO 影像上。①是偵測器的原始輸出（把 NMS 關掉）：同一批物件上疊了 53 個框，這是因為相鄰的預測點都看見了同一個東西、各自報了一個框。②是第一輪：綠色是全場信心最高的框，灰色那 9 個與它的 IoU 超過 0.5，被視為重複而刪除。③是把這個程序做到底的結果：53 個框收斂成 7 個。右圖是門檻的影響——門檻從 0.1 調到 1.0，保留的框從 4 個增加到 53 個。這裡有個容易混淆的地方值得點破：NMS 的 IoU 門檻與評分用的 IoU 門檻（圖 16-8）是兩回事。前者問「這兩個預測是不是在講同一個物件」，後者問「這個預測跟標準答案夠不夠接近」。它們剛好都叫 IoU、也剛好常常都設 0.5，但調錯了哪一個，後果完全不同。另外請記住 16.3.2 節提到的同名之緣：這裡的「非極大值抑制」與第 9 章 Canny 邊緣偵測的第三步是同一個思想——沿著某個方向只留下最強的那一個，把它的鄰居壓掉。

圖16-6　NMS：一步一步看它刪掉什麼

16.3.3 評估指標：mAP

物件偵測的標準評估指標是 mAP(mean Average Precision，平均精度均值)。它建立在第 14 章的精確率與召回率之上：對每一個類別，調整信心門檻可得到一條「精確率-召回率曲線」，曲線下的面積即為該類別的 AP;把所有類別的 AP 取平均，就是 mAP。常見的寫法如 mAP@0.5(以 IoU 門檻 0.5 判定正確)或 mAP@0.5:0.95(在多個 IoU 門檻下取平均，標準更嚴格，是 COCO 資料集的主要指標)。

請留意這裡與第 14 章的連貫：同樣是「精確率 vs. 召回率」的權衡，同樣是「依應用代價選擇門檻」的工程判斷。一個工安監控系統可能寧可誤報也不能漏報(重召回)，一個自動分揀系統則可能相反(重精確)。指標的選擇從來不只是數學問題。



16.3.3 節說 mAP「對每一個類別調整信心門檻可得到一條精確率-召回率曲線，曲線下面積即 AP，所有類別取平均即 mAP」。這張圖把那句話拆開。左、中兩圖是兩個真實類別的曲線（person、car）。藍線是原始曲線，它上上下下——因為每多接受一個預測，精確率可能升也可能降。綠色虛線是「單調化」之後的版本：把每一點的精確率換成它右邊所有點的最大值，再積分那塊面積，這就是 COCO 採用的全點內插 AP。為什麼要單調化?因為實務上你總能選一個更好的門檻，所以曲線上的凹陷不該算進成績。person 的 AP 是 0.792（若改用 2007 年 VOC 的十一點內插法則是 0.767，同一條曲線、不同算法，差了 2.5 個百分點——這是比較論文數字時務必確認的細節）。右圖是 mAP 的「m」：保留集上有標註的 79 個類別，各自的 AP 從 99% 到 0% 不等，平均起來才是 56.5%。順帶注意右圖那幾個 100%：它們各自只有 12 到 28 個標註，樣本這麼少時「滿分」沒什麼意義（第 14.5.3 節的標準誤公式）;真正撐起這個 mAP 的是 person 那種有 1284 個標註的類別。這張圖最該帶走的一句話是：mAP 是一個平均，而平均會藏住災難——如果你真正在乎的那一類剛好落在最差的那五名，再漂亮的 mAP 也救不了你。這與第 14 章「看混淆矩陣而不是看平均」是同一課。

圖 16-7　AP 就是那塊面積

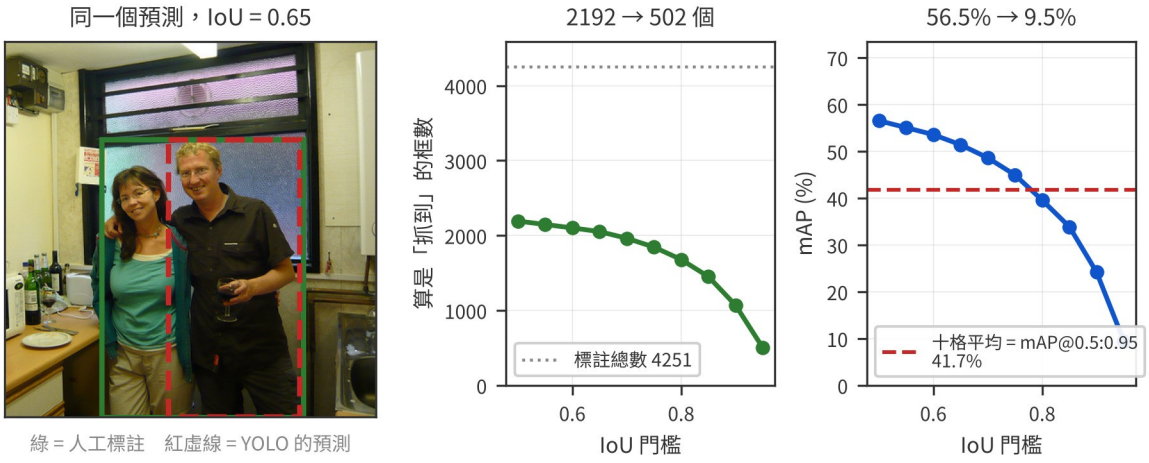
16.3.4 推導：IoU、NMS 與 mAP 的三道算式

16.3 節用文字介紹了三個關鍵概念，它們其實都只是幾行算術。這一小節把它們寫成可以照著實作的式子——本章所有圖裡的數字，就是用這幾行算出來的，而且與 Ultralytics 官方工具的結果只差一個百分點左右(差異來自群體標註與面積過濾等細節規則)。親手算過一遍，你對那些分數的信任才會建立在理解之上，而不是建立在別人的函式庫之上。

先看 IoU。兩個框各用左上與右下兩點表示為 (x1, y1, x2, y2)，交集矩形的左上角取兩者較大的，右下角取兩者較小的：

```
ix1 = max(a.x1, b.x1)       iy1 = max(a.y1, b.y1)
ix2 = min(a.x2, b.x2)       iy2 = min(a.y2, b.y2)
交集 = max(0, ix2 − ix1) × max(0, iy2 − iy1)
聯集 = 面積(a) + 面積(b) − 交集
IoU  = 交集 / 聯集
```

那兩個 max(0, ...) 是整段程式唯一需要小心的地方：兩個框完全不相交時，ix2 − ix1 會是負數，少了這道截斷就會算出一個莫名其妙的正面積。另外注意聯集要減掉交集，否則重疊的部分被算了兩次。從這個式子可以立刻讀出兩件圖 16-5 量到的事：其一，IoU 對平移的敏感度與框的大小無關，只與「偏移佔邊長的比例」有關;其二，若兩個同中心的正方形邊長比是 s(s > 1)，交集就是小的那個、聯集是大的那個，於是 IoU = 1/s²，令它等於 0.5 得 s = √2 ≈ 1.41——這正是圖 16-5 實測到的上界 1.41 倍，而縮小的那一側同理是 1/√2 ≈ 0.71。



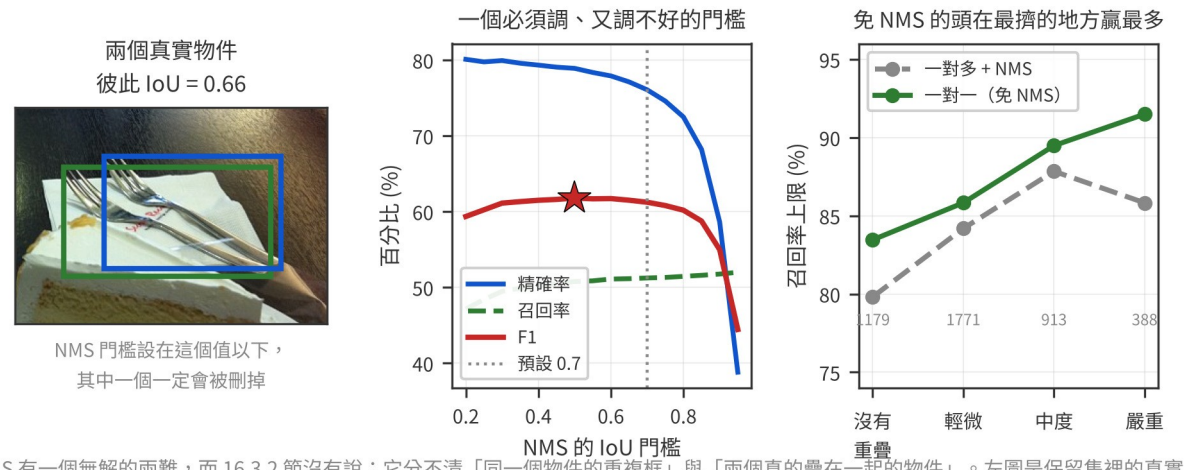
上一張圖量了 IoU 的幾何，這一張量它對成績的影響。左圖是一個真實的例子：YOLO 對這個物件給出的框與人工標註的 IoU 是 0.65，看起來相當準——但在 IoU 0.5 的門檻下它是命中，在 0.75 的門檻下它就變成一次漏抓加一次誤報。同一個預測、同一張圖，換個門檻就換個身分。中圖把 600 張圖的 4251 個標註框全部算一遍（信心門檻固定 0.25）：門檻 0.5 時有 2192 個算抓到，門檻 0.95 時只剩 502 個，掉到 23%。右圖是 mAP 的同一條線：56.5% 一路降到 9.5%。請注意這條曲線在 0.5 到 0.7 之間還算平緩，過了 0.8 就陡降——那是因為要把框畫到 IoU 0.9 以上，等於要求邊界誤差小於邊長的 5%（圖 16-5），而人工標註本身都未必有這個精度。COCO 的主指標 mAP@0.5:0.95 就是這十格的平均（紅色虛線），本圖實測為 41.7%，遠低於大家習慣引用的 mAP@0.5 的 56.5%。兩個數字都沒有錯，但它們回答的是不同的問題：mAP@0.5 問「有沒有找到東西」，mAP@0.5:0.95 問「框畫得準不準」。報告成績時務必寫清楚是哪一個，否則比較毫無意義。

圖 16-8　IoU 門檻決定什麼算「抓到」

再看 NMS。它是一個貪婪演算法，只有四行：

```
把所有框依信心分數由高到低排序
while 還有框:
    取出分數最高的框，放進保留清單
    把剩下的框中，與它 IoU > 門檻 的全部刪除
```

這裡有一個常被忽略的細節：實務上 NMS 應該「逐類別」進行。一個人站在腳踏車前面，兩個框可能高度重疊，但它們講的是不同的東西，不該互相抑制。Ultralytics 的做法是先把每個框的座標按類別編號平移一段很大的距離(agnostic=False 時)，讓不同類別的框在幾何上永遠不可能相交，再跑一次全域的 NMS——一個很漂亮的小技巧。另外值得估一下成本：最壞情況下每一輪要與剩下的框各算一次 IoU，總共 $O(n^2)$ 次;n 是 NMS 前的框數，而圖 16-6 的例子中 n 是 55，若把信心門檻設成 0.001(算 mAP 時必須這麼做)，n 可能上千，這時 NMS 本身就會變成瓶頸。這也是實務上總會先用信心門檻砍一刀、再限制每張影像最多保留幾個框(Ultralytics 預設 300)的原因。



NMS 有一個無解的兩難，而 16.3.2 節沒有說：它分不清「同一個物件的重複框」與「兩個真的疊在一起的物件」。左圖是保留集裡的真實例子——兩個同類物件的真實框彼此 IoU 就有 0.66。只要 NMS 門檻設在這個值以下，其中一個必定被當成重複而刪除，而且是在偵測器正確找到它之後才刪的。中圖把整個保留集掃一遍：門檻從 0.2 調到 0.95，召回率從 47.1% 升到 52.0%，精確率卻從 80.1% 掉到 38.8%；F1 在 0.50 處最高，但那個最佳值完全取決於場景有多擠——這是一個必須調、又沒有通用答案的門檻。右圖是 YOLO26 給的新答案。它的「一對一」預測頭設計成每個物件只報一個框，因此根本不跑 NMS，那個門檻也就不存在了。把兩個頭放到能力上限（信心門檻 0.001）比較，按「該框與鄰居的最大 IoU」分組：完全不重疊時兩者差 +3.6 個百分點，輕微 +1.6、中度 +1.6，而重疊嚴重（IoU > 0.35）時免 NMS 的頭以 91.5% 對 85.8%，領先 5.7 個百分點——優勢正好隨著場景變擠而變大。這很合理：NMS 愈是在重疊處誤刪，拿掉它的收穫就愈大。所以人群計數、貨架補貨、密集果實採收、停車場車位這類場景，優先考慮免 NMS 的預測頭。但也別把它當萬靈丹：它解掉的是「後處理誤刪」，不是「遮擋本身讓物件難認」（圖 16-17 會把這兩件事分開量）;而且在不重疊的情況下兩者差距很小，換過去主要是為了部署單純。

圖 16-9　NMS 的兩難，以及 YOLO26 怎麼把它拿掉

最後是 AP。它分成三步：配對、累積、積分。配對的規則是：把預測依分數由高到低處理，每個預測找 IoU 最大且尚未被認領的同類真實框;IoU 超過門檻才算命中(TP)，否則算誤報(FP)，而且每個真實框最多被認領一次——這條「最多一次」的規則就是為什麼重複的框會變成誤報，也是 NMS 必須存在的理由。接著沿著分數由高到低累積：

```
召回率 R(k) = 前 k 個預測中的 TP 數 / 真實框總數
精確率 P(k) = 前 k 個預測中的 TP 數 / k
AP = ∫ P dR    (曲線下面積)
```

直接積分會遇到一個問題：精確率曲線是鋸齒狀的，每多接受一個預測，精確率可能升也可能降(圖 16-7 的藍線)。標準做法是先讓它單調不升——把每一點的精確率換成它右邊所有點的最大值：

```
P*(R) = max{ P(R') : R' ≥ R }
AP = Σ (R(i+1) − R(i)) × P*(R(i+1))
```

這個「單調化」的理由是：既然你總能選一個更好的門檻，那麼曲線上的凹陷就不該算進成績。歷史上還有另一種算法——2007 年 PASCAL VOC 的十一點內插法，只在召回率 0、0.1、…、1.0 這十一個點取精確率再平均。兩者用同一條曲線可以差好幾個百分點(圖 16-7 實測 person 類別：全點 0.782、十一點 0.758)，所以比較不同論文的數字時，先確認它們用的是同一種算法。

把 AP 對所有類別取平均得到 mAP;再把 IoU 門檻從 0.50 掃到 0.95(每 0.05 一格)把十個 mAP 平均起來，就是 COCO 的主指標 mAP@0.5:0.95。圖 16-8 畫出了那十

個數字，而它們的差距之大(從 56.5% 到 9.5%)正好說明：沒有指明門檻的 mAP，是一個沒有意義的數字。

表 16-1 物件偵測的關鍵概念

16.4 YOLO 實戰：推論、訓練與驗證

16.4.1 環境安裝與快速推論

Ultralytics 把 YOLO 的使用門檻降到極低。安裝一個套件、載入預訓練模型，幾行程式就能偵測 COCO 資料集(第 14 章)的八十個類別——人、車、貓、狗、瓶子、椅子等日常物件。

程式 16-1 安裝 Ultralytics YOLO

```
# 安裝(建議在虛擬環境中)
pip install ultralytics
```

程式 16-2 YOLO 影像推論

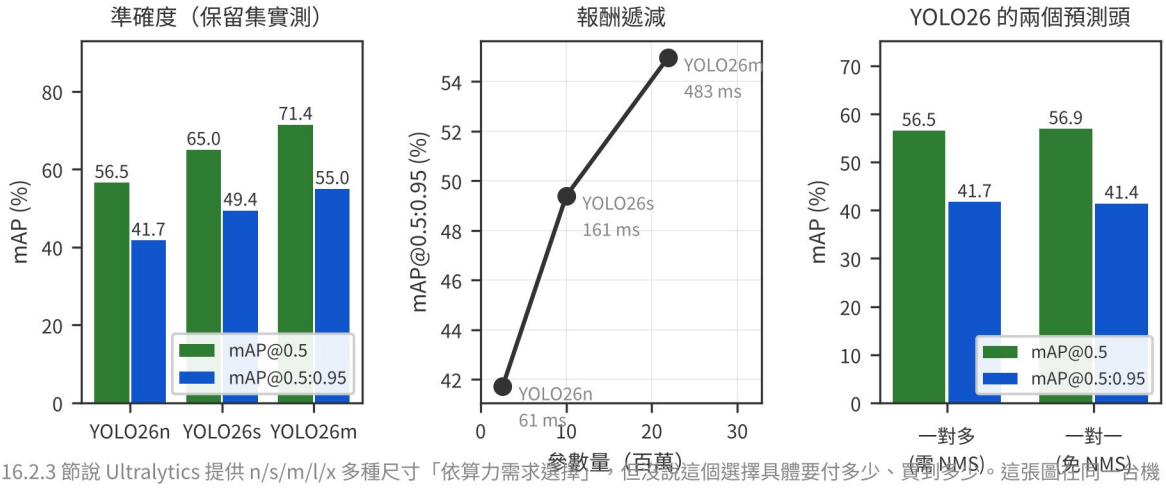
```
from ultralytics import YOLO
import cv2

# 載入預訓練模型(n/s/m/l/x 依算力需求選擇)
model = YOLO("yolo26n.pt")        # nano:最輕量,適合邊緣裝置

# --- 單張影像推論 ---
results = model("street.jpg", conf=0.25, iou=0.45)
#   YOLO26 另有免 NMS 的端到端模式(16.2.4 節):
#   results = model("street.jpg", conf=0.25, nms=False)
#   conf:信心門檻(調高減少誤報、調低減少漏檢)
#   iou :NMS 的 IoU 門檻(用 nms=False 時此參數不生效)

for r in results:
    for box in r.boxes:
        cls  = int(box.cls[0])                    # 類別編號
        name = model.names[cls]                   # 類別名稱
        conf = float(box.conf[0])                 # 信心分數
        x1, y1, x2, y2 = box.xyxy[0].tolist()    # 邊界框座標
        print(f"{name}: {conf:.2f} @ ({x1:.0f},{y1:.0f})-({x2:.0f},
{y2:.0f})")

    annotated = r.plot()          # 畫出所有偵測框
    cv2.imwrite("detected.jpg", annotated)
```



16.2.3 節說 Ultralytics 提供 n/s/m/l/x 多種尺寸「依算力需求選擇」，但沒說這個選擇具體要付多少、賺到多少。這張圖在同一台機器上量給你看，並且多問一個 YOLO26 才有的問題：要不要 NMS。尺寸方面，從 n 到 m，mAP@0.5 從 56.5% 升到 71.4%，mAP@0.5:0.95 從 41.7% 升到 55.0%，多了 13.3 個百分點;代價是參數從 2.6 百萬變成 21.9 百萬、單張耗時從 61 毫秒變成 483 毫秒（7.9 倍），在這台 CPU 上等於從 16 fps 掉到 2.1 fps。中圖畫出這條報酬遞減曲線：n 到 s 這一步最划算，s 到 m 就明顯平了。右圖是 YOLO26 的新選項。它同時訓練兩個預測頭：預設的「一對多」頭讓多個預測點各報一個框，因此必須靠 NMS 去重（圖 16-6）;而「一對一」頭（推論時加上 nms=False）設計成每個物件只報一個框，完全不需要 NMS，輸出固定是 300 個框。實測兩者的準確度幾乎相同：mAP@0.5 是 56.5% 對 56.9%、mAP@0.5:0.95 是 41.7% 對 41.4%——也就是說，免 NMS 這件事幾乎是白送的。要誠實說明的是：本圖在 CPU 上量不出明顯的速度差，因為 PyTorch 後處理只佔總時間的百分之一上下;Ultralytics 官方報告的「CPU ONNX 快上四成」是在匯出成 ONNX 並在特定伺服器 CPU 上量的，條件與這裡不同，不宜直接套用。免 NMS 真正的價值在別處：整個模型可以匯出成單一張計算圖，不必在 TensorRT、LiteRT 或瀏覽器端另外實作一段後處理;輸出形狀固定，對只吃靜態形狀的推論引擎特別友善;而且少了一個必須依場景調的 NMS 門檻（圖 16-9 會量這件事值多少）。這三點對第 17 章的邊緣部署都是決定性的差別。該怎麼選?規則很簡單：先問應用需要幾 fps，那是一條硬線;在這條線以內挑能跑的最大模型，並且只要部署端不方便寫後處理，就優先用免 NMS 的那個頭。

圖 16-10　模型尺寸與兩個預測頭：要不要 NMS

16.4.2 訓練自訂資料集

真正的價值在於訓練自己的偵測器——偵測您關心的物件(工件瑕疵、農產品、特定零件)。流程分三步：

1. 準備資料：蒐集影像並標註邊界框(常用工具如 LabelImg、Roboflow、CVAT)。標註格式為 YOLO 格式：每張影像對應一個 txt 檔，每行記錄「類別編號 中心 x 中心 y 寬 高」(座標皆正規化到 0 至 1)。務必依第 14 章原則切分訓練/驗證/測試集。
2. 撰寫設定檔：一個 YAML 檔，指定資料集路徑與類別名稱。
3. 訓練與驗證：呼叫 train() 開始訓練(內建資料增強、及早停止等第 14、15 章的機制)，再用 val() 在驗證集上計算 mAP。

程式 16-3 資料集設定檔

```
# data.yaml — 資料集設定檔
path: /home/user/mydata
train: images/train
val:   images/val
names:
  0: good_part
  1: scratch
  2: crack
```

程式 16-4 訓練自訂 YOLO 模型

```
from ultralytics import YOLO

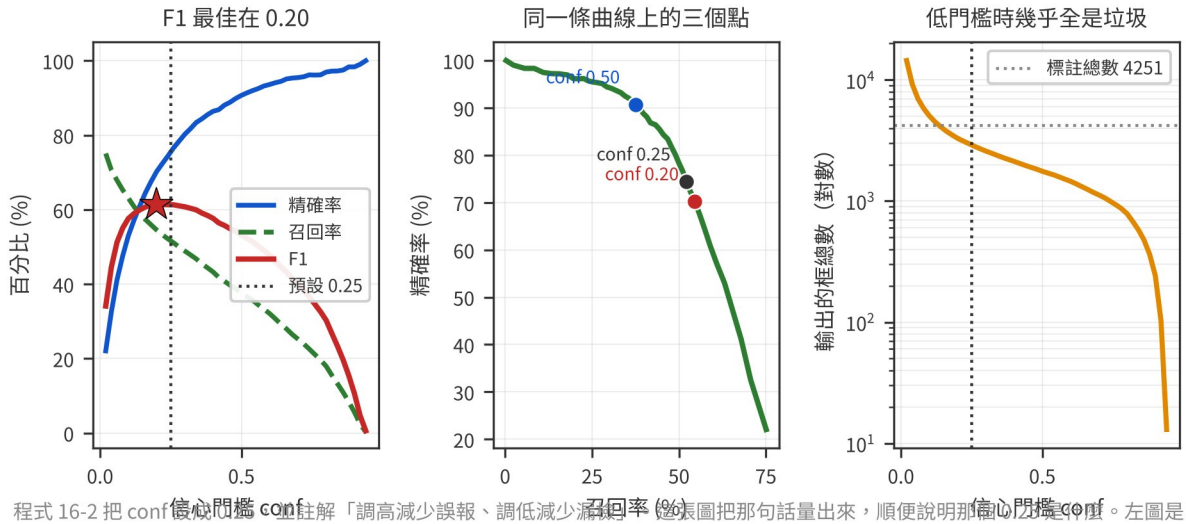
# 從預訓練權重開始訓練 = 遷移學習(第 15 章)
model = YOLO("yolo26n.pt")

results = model.train(
    data="data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    patience=20,           # 及早停止(第 14 章對抗過度擬合)
    degrees=10,            # 資料增強:旋轉(第 4 章)
    hsv_v=0.4,             # 資料增強:亮度(第 5、8 章)
    fliplr=0.5,            # 資料增強:水平翻轉(第 4 章)
    project="runs", name="defect_v1")

# 驗證:計算 mAP
metrics = model.val()
print("mAP@0.5      :", metrics.box.map50)
print("mAP@0.5:0.95 :", metrics.box.map)

# 匯出為部署格式(第 17 章邊緣部署)
model.export(format="onnx", nms=False)   # 免 NMS:匯出成單一張計算圖
#   也可用 engine(TensorRT)、tflite(LiteRT)、openvino、coreml
```

請注意程式 16-4 中的訓練參數：degrees、hsv_v、fliplr 全是第 4、5、8 章的影像處理技術，在此作為資料增強使用;patience 是第 14 章的及早停止;而「從預訓練權重開始」正是第 15 章的遷移學習。這再次印證全書的主軸——最先進的工具，骨子裡仍是前面各章觀念的組合。



程式 16-2 把 conf 設成 0.25 並註解「調高減少誤報、調低減少漏報」，這張圖把那句話量出來，順便說明那個門檻到底是什麼。左圖是三條線隨門檻的變化（600 張保留影像、4251 個標註框、IoU 0.5 判定）：門檻 0.02 時召回率 75.1%、精確率只有 22.1%;門檻 0.94 時精確率升到 100.0%、召回率只剩 0.3%。F1 的最佳點落在 0.20，而預設的 0.25 給出的是精確率 74.4%、召回率 52.2%、F1 61.3%。中圖把同一件事畫成精確率-召回率曲線：調 conf 不會讓模型變好，只是讓你沿著這條曲線移動——這是第 14.5 節那一課在偵測上的翻版。右圖說明為什麼低門檻不能用：門檻 0.02 時輸出 14,469 個框，而真實物件只有 4251 個，也就是說超過 71% 是誤報。那麼 0.25 是怎麼來的?在這份資料上它其實相當接近 F1 的最佳點（0.20，差 0.1 個百分點），所以它不是隨便訂的——但它最佳化的是 F1，也就是「把精確率與召回率看得一樣重」。你的應用多半不是這樣。工安監控寧可誤報也不能漏報，門檻該調低;自動分揀誤判就是整批重工，門檻該調高。還有一個陷阱要記住：算 mAP 時 conf 必須設到極低（本章用 0.001），因為 AP 要涵蓋整條曲線;若先用 0.25 砍掉低分預測，曲線的右端就永遠補不回來，mAP 會被低估。

圖 16-11　信心門檻：那個被當成預設值的 0.25

16.4.3 推導：該標幾張、該訓練多久、該相信幾分

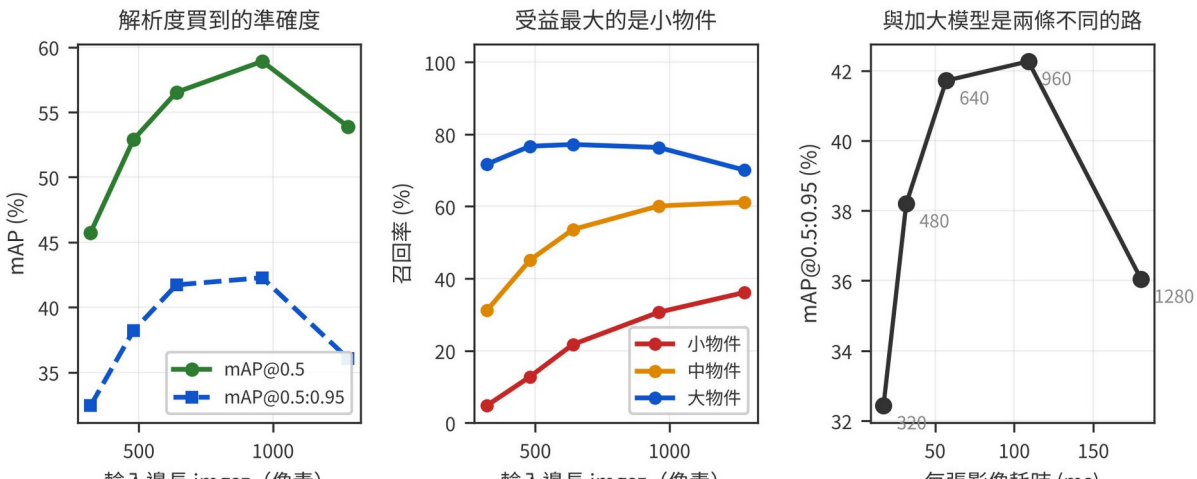
16.4.2 節把訓練自訂偵測器寫成三個步驟，但學生最常問的三個問題它都沒回答：要標幾張才夠?要訓練幾個 epoch?最後測出來的 mAP 可以相信幾分?這三個問題都沒有唯一答案，但都有可以估算的下界，而估算的工具在第 14 章都給過了。

先談最後一個，因為它決定前兩個。第 14.5.3 節推過：在 n 筆測試樣本上量到的比例 a，其標準誤約為 $\sqrt{a(1-a)/n}$。偵測問題的單位不是影像而是「物件」，所以 n 要用標註框數來算。假設你的驗收集有 200 個標註框、量到的召回率是 0.90，那麼標準誤是 $\sqrt{0.9 \times 0.1 / 200} \approx 0.021$，95% 區間大約是 90% ± 4 個百分點。換句話說用 200 個框去驗收，你分不出 88% 與 92% 的差別。

mAP 的情況更糟，因為它是「先逐類別算 AP、再平均」。一個只有三個標註框的類別，它的 AP 只有少數幾種可能值，平均進去就是在擲骰子——圖 16-18 實測保留集的 79 個類別中，標註數不到四個的那幾類，AP 幾乎都是 0% 或 100%。同一張圖用重抽樣量出：只用其中 128 張時，mAP@0.5 的標準差有 2.8 個百分點，用滿 600 張才降到 1.1。可操作的規矩是：你真正在乎的每一個類別，驗收集至少要有幾十個標註框;少於十個的類別，不要報它的 AP，改報「抓到幾個、漏掉幾個」這種可以直接數的量。

回到第一個問題：訓練集要標幾張?這要看任務有多難、以及你從哪裡開始。程式 16-4 的第一行就是答案的一半——從 yolo26n.pt 這個 COCO 預訓練權重開始，就是第 15 章的遷移學習。圖 15-18 量過它的規律：資料愈少，借來的特徵愈划算;資料多到足以自己學，凍結反而變成天花板。偵測任務的經驗起點是「每類一百到數百張、每張數個物件」，但真正該做的不是背這個數字，而是畫一條學習曲線：用 25%、50%、100% 的資料各訓練一次，看驗證 mAP 有沒有還在上升。還在上升就代表加資料有用，已經平掉就代表瓶頸在別處(多半是標註品質，見圖 16-13，或是類別本身太難)。

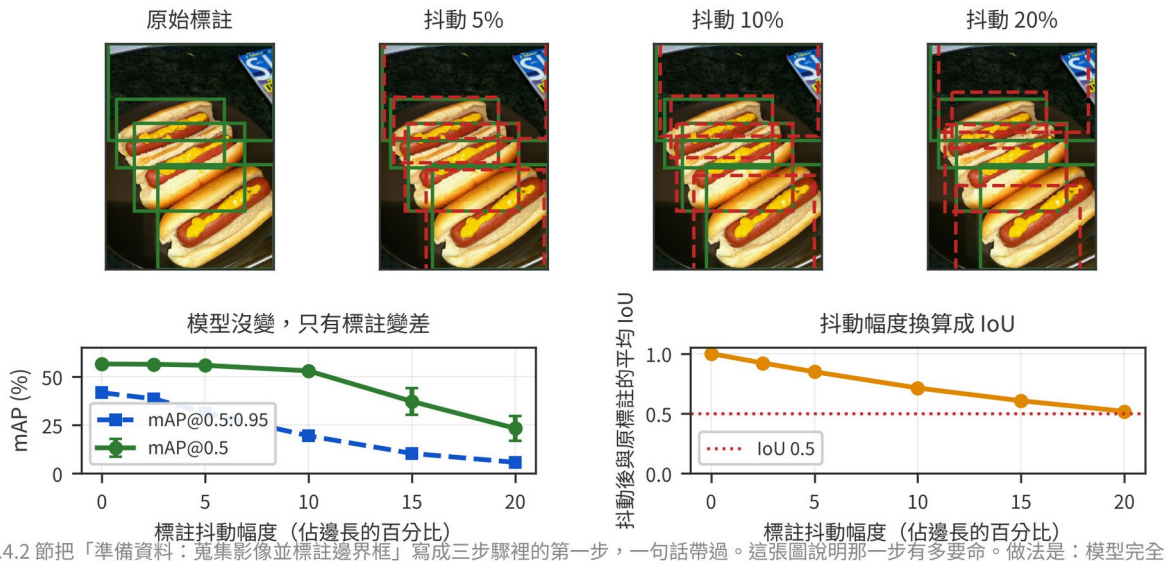
第二個問題：訓練幾個 epoch?程式 16-4 寫 epochs=100、patience=20，這兩個數字要一起讀：真正決定何時停的是 patience，epochs 只是上限。它的意思是「驗證指標連續 20 個 epoch 沒有進步就停」，正是第 14.6.3 節的及早停止。所以 epochs 設大一點並不浪費(大不了提早停)，而 patience 設太小才危險——訓練曲線本來就會上下抖動，patience 太小會在一次正常的抖動中誤判為「不再進步」。



程式 16-4 把 imgsz 設成 640 就沒再提它，但在小物件問題上（圖 16-4），這個旋鈕比換模型還有效——只是它的行為跟多數人以為的不一樣，所以這張圖量了五種解析度。先看中圖，那是符合直覺的部分：輸入從 320 加到 1280，小物件的召回率從 5% 一路升到 36%，翻了 7.5 倍。道理很單純：一個在原圖上只有 20 像素的物件，縮到 320 之後可能只剩幾個像素，連 stride 8 的格子都裝不下它（圖 16-2）;把輸入放大，它才重新變成「看得見」的東西。但同一張圖也說了反直覺的那一半：大物件的召回率在 640 時最高（77%），到 1280 反而掉到 70%。於是左圖的總成績並不是一路向上，而是一條倒 U：mAP@0.5 在 imgsz = 960 達到最高的 58.9%，320 時只有 45.7%，1280 時又掉回 53.9%。原因不難想：YOLO26 的預訓練權重是在 640 上訓出來的，把輸入放大一倍，所有物件在網路眼中都大了一倍，大物件因此被推出了它熟悉的尺度範圍——這就是第 15 章說的分布偏移，只是發生在尺度這個維度上。右圖是代價：耗時從 17 毫秒增加到 180 毫秒，大致隨像素數成正比，所以 1280 是「花了 3.2 倍的時間換到更差的總成績」。三個可以直接拿去用的結論。其一，調 imgsz 與換模型（圖 16-10）是兩條不同的路：加大模型是讓網路更會認，加大輸入是讓小東西還在。其二，推論時的 imgsz 不要離訓練時的值太遠;真的需要高解析度，正確做法是用高解析度「重新訓練」，而不是只在推論時放大。其三，若漏抓集中在小物件而又不能重訓，把大圖切塊、各自以 640 偵測再合併，通常比直接放大整張圖有效。

圖 16-12　解析度：imgsz 這個被忽略的旋鈕

最後補一個偵測特有、而且非常容易踩的陷阱：切分不能只看影像。同一條產線同一批果實、同一段影片的相鄰影格，彼此極度相似;若把它們隨機打散後再切訓練與驗證集，驗證集裡就會出現「訓練集那張照片的孿生兄弟」，分數會虛高好幾個百分點。這是第 14 章資料洩漏的偵測版本。正確做法是按「來源」切——依拍攝日期、依批次、依影片檔，讓同一個來源的影像整組落在同一邊(scikit-learn 的 GroupKFold 就是為此而生)。這一條比任何調參都重要：切分錯了，後面量到的每一個數字都不必看了。



16.4.2 節把「準備資料：蒐集影像並標註邊界框」寫成三步驟裡的第一步，一句話帶過。這張圖說明那一步有多要命。做法是：模型完全不動，只把人工標註隨機抖一抖（平移與縮放，幅度以邊長的百分比計），再重算一次成績。上排是抖動長什麼樣——抖動 5% 時肉眼幾乎看不出差別。但成績會：抖 5% 時 mAP@0.5 從 56.5% 掉到 55.8%（少 0.7 分），mAP@0.5:0.95 從 41.7% 掉到 31.4%（少 10.3 分）;抖 20% 時更掉到 23.3% 與 5.7%。請特別注意兩條線的斜率差：mAP@0.5:0.95 掉得明顯更快，因為它本來就在量「框準不準」，而標註歪了，準的定義就歪了。右圖把抖動換算成 IoU：抖 10% 時，抖動後的框與原標註平均 IoU 還有 0.71——在 0.5 的門檻下它們「是同一個框」，成績卻已經少了好幾分。三個實務結論。其一，標註品質是成績的天花板，換模型換不回來。其二，比較兩個模型時，兩邊必須用同一份標註，否則你比的是標註員而不是模型。其三，驗收標準要跟著標註精度走——如果你的標註本身只有 IoU 0.9 的一致性，那就不該拿 mAP@0.95 當 KPI。這是第 14 章「誠實地知道模型有多好」在偵測上的具體版本。

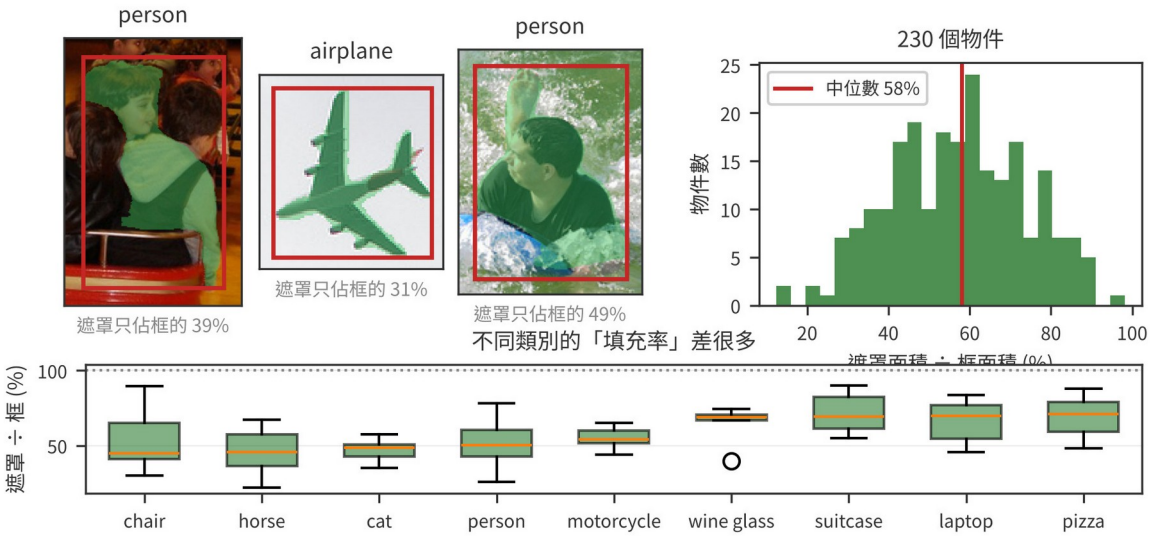
圖 16-13　標註品質：框畫歪一點，成績掉多少

16.5 延伸功能：分割、姿態估計與追蹤

現代 YOLO 已不只是偵測器，而是一套整合的視覺工具箱，只需更換模型權重即可切換任務。

16.5.1 實例分割

使用 seg 模型，YOLO 除了邊界框還會輸出每個物件的像素級遮罩(mask)，達成 16.1 節所述的實例分割。這比單純的框更精確——量測物件的實際面積、計算不規則形狀的尺寸、或做背景去除時，遮罩遠優於矩形框。它也是第 9 章傳統分割方法的深度學習後繼者。

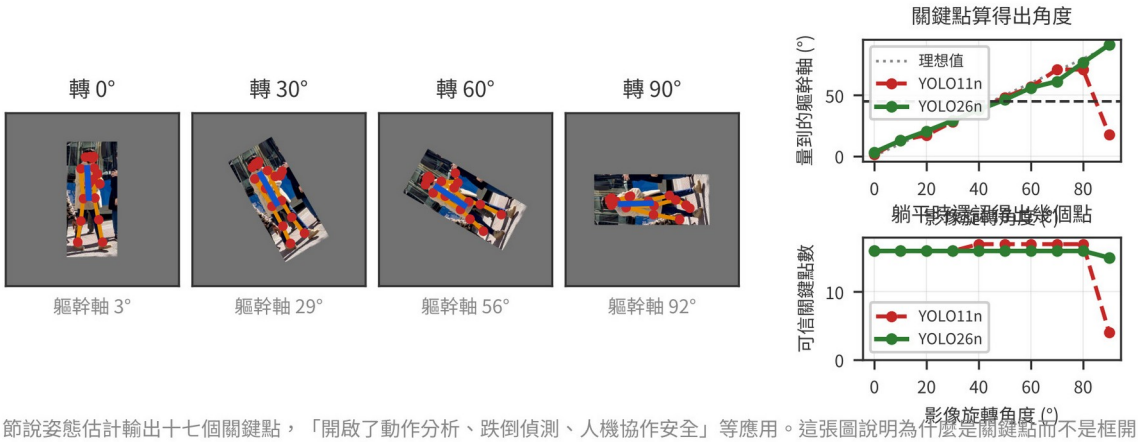


16.5.1 節說分割模型「除了邊界框還會輸出像素級遮罩，這比單純的框更精確——量測物件的實際面積」。這張圖量出「更精確」是多少。做法是對保留集跑一次 YOLO26n-seg，把每個物件的遮罩面積與它自己的框面積相除，得到一個「填充率」：如果物件是個實心矩形，這個值會是 100%。實測 230 個物件的中位數只有 58%——也就是說，用框去估面積平均會高估約 1.7 倍。上排三個例子說明原因：細長、彎曲或張開的東西（圖中這三個的填充率只有 39%、31%、49%）外接矩形裡裝的多半是背景。下圖更要緊：填充率是「看類別」的。形狀接近方塊的東西填充率高，細長或鏤空的東西低，而且同一類內部的離散程度也不小。這對實務的意義很直接：只要你的應用要「量」而不只是「找」——果實大小分級、瑕疵面積佔比、藥片計數與體積估計、畜牧的體態評分——框就不夠用，必須上分割。反過來說，如果只是計數或定位，框便宜得多（標註快、模型小、後處理簡單），沒必要為了好看而上分割。另外提醒一件標註成本的事（圖 16-2 已經量過輸出規模）：描一個輪廓的時間是畫一個框的好幾倍，所以「要不要分割」往往不是技術問題，是預算問題。

圖 16-14　框與遮罩：量面積時差多少

16.5.2 姿態估計

使用 pose 模型，YOLO 會輸出人體的關鍵點(通常十七個：鼻、雙眼、雙耳、雙肩、雙肘、雙腕、雙髖、雙膝、雙踝)，形成骨架。這開啟了豐富的應用：動作分析與運動姿勢矯正、人機協作中判斷作業員的手部位置以確保安全、手勢指令控制、以及第 12 章討論過的跌倒偵測——透過追蹤重心高度的急遽變化與骨架的姿態，能比單純的框更可靠地判斷跌倒。

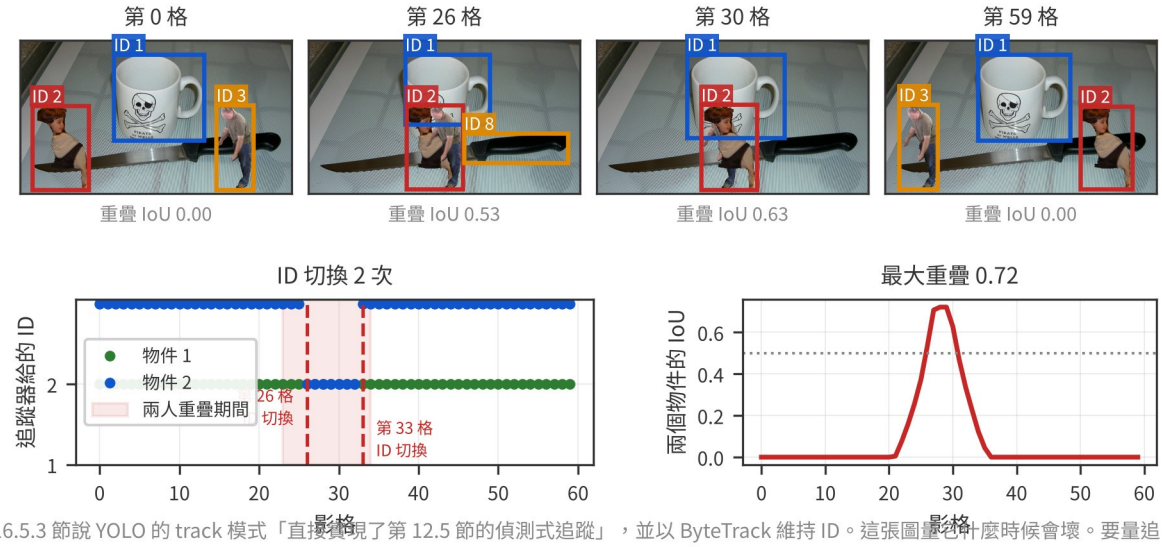


16.5.2 節說姿態估計輸出十七個關鍵點，「開啟了動作分析、跌倒偵測、人機協作安全」等應用。這張圖說明為什麼是關鍵點而不是框開啟了這些應用，並且比較兩代模型在最關鍵的那個角度上的差別。做法是把同一個人的影像逐步旋轉（模擬人倒下），每一步重做一次估計，再取雙肩中點與雙髖中點連成「軀幹軸」（Lab 16-3 要做的事），算它與垂直線的夾角。YOLO26n 量到的角度從 3° 一路跟到 92°，幾乎貼著理想的對角線——它是一個可以直接設門檻的連續量，超過 45° 就報警。相較之下，只看偵測框的寬高比從 0.37 變到 2.51，方向雖對卻容易被污染：人蹲下、彎腰、張開雙臂，寬高比都會變，而軀幹軸不會。這就是「輸出結構」的價值（圖 16-1 量過它的資訊量）。右邊兩張圖則是這一版真正的進步所在。上一代的 YOLO11n 在轉到 90°、也就是人完全躺平時整個崩掉：可信關鍵點從 17/17 掉到 4/17，軀幹軸算出 18°——一個完全錯誤、而且看起來像「站著」的值。YOLO26n 在同一個角度仍有 15/17 個可信關鍵點，軀幹軸 92°，判斷正確。這與官方宣稱姿態估計 AP 提升的方向一致，而且提升的正是最需要它的那一端。話雖如此，16.6.4 節那句「不能單靠一個可能誤判的 AI 模型」依然成立：本圖只旋轉了一個人、一種背景，真實的跌倒還有遮擋、拍攝角度、衣著與光線的變化。正確的系統設計仍是把姿態估計當成第一道預警、要求連續多格一致、並搭配其他感測器——下一步的算術在圖 16-19。

圖 16-15　姿態估計：十七個點，與兩代模型在躺平時的差別

16.5.3 物件追蹤

YOLO 內建追蹤功能，直接實現了第 12.5 節介紹的「偵測式追蹤」(tracking-by-detection)：每一影格用 YOLO 偵測，再以 ByteTrack、BoT-SORT 等演算法把跨影格的偵測結果關聯起來、賦予穩定的 ID。這正是第 12 章所說 SORT/DeepSORT 框架的現代實作。有了穩定的 ID，才能做人流計數、車流分析、軌跡記錄——這些在第 12.7 節的案例中都已鋪陳過，現在終於能用最先進的工具實現。



16.5.3 節說 YOLO 的 track 模式「直接實現了第 12.5 節的偵測式追蹤」，並以 ByteTrack 維持 ID。這張圖量它什麼時候會壞。要量追蹤就得有正確答案，所以這裡用真實素材合成一段有答案的序列：從 COCO 影像中用分割遮罩摳出兩個人，貼到一張真實背景上，讓他們以固定速度交錯而過，共 60 格（第 12 章那一套「自己造一個有正確答案的序列」的做法）。上排四格是結果：交會前兩人各有穩定的 ID，交會時兩個框重疊到 IoU 0.72，交會後再看它們的 ID 還對不對。左下是完整的 ID 軌跡圖，實測整段序列發生 2 次 ID 切換，而且發生的位置完全不意外：第 26 格（重疊 IoU 0.53，正是交會最深的那一刻）物件 1 的 ID 從 3 跳到 2，第 33 格兩人分開後又跳了回來。值得注意的是偵測本身沒有失敗——兩個物件在全部 60 格都有框對應到（60 與 60 格），壞掉的只有「誰是誰」這件事。為什麼交會是追蹤的死穴?因為偵測式追蹤的關聯靠的是「這一格的框跟上一格哪個框最像」，而兩人重疊時，框的位置、大小、甚至外觀都彼此相似，幾何線索暫時失效;更糟的是 NMS 可能在此時把其中一個整個刪掉（圖 16-9）——沒有偵測，就沒有東西可以關聯。實務上的對策有三層：提高偵測的召回（調 NMS 與 conf）、換用會記外觀特徵的追蹤器（如 BoT-SORT 的 ReID）、以及在應用層容忍——例如車流計數只在固定的計數線上判斷，就算 ID 中途換過，越線的那一瞬間仍然數得對。Lab 16-4 要做的正是這種設計。

圖 16-16　偵測式追蹤：ID 在交會時還認得出來嗎

16.5.4 YOLO26 再擴充的任務家族

前面三小節的分割、姿態與追蹤，從 YOLOv8 時代就有了。YOLO26 又補上三種任務與一個特別的變體，它們各自回答一個框回答不了的問題。用法一律是換一個權重檔，介面完全相同。

語意分割(yolo26n-sem.pt)為畫面上每一個像素標類別，但不區分同類的不同個體——這正是 16.1 節區分過的第三種任務，也是第 9.6 節 U-Net 擅長的那一類。它與實例分割的分工很清楚：要數「有幾顆果實、每顆多大」用實例分割;要問「這張空拍圖裡農田、道路、建物各佔多少面積」——個體根本無從數起——就用語意分割。農地分區、道路可行駛區域、瑕疵區域佔比，都屬於後者。

單眼深度估計(yolo26n-depth.pt)從一張普通的彩色影像推估每個像素的相對深度。請特別注意「單眼」與「相對」這兩個限制詞。單眼表示它不靠雙目視差、也不靠光達，而是從透視、遮擋、紋理梯度這些線索推論——這與第 4 章談相機模型時說過的「單張影像遺失了深度維度」是同一件事，所以它推出來的是一種合理的猜測，不是量測。相對則表示輸出沒有絕對尺度：它能告訴你 A 比 B 近，卻不能直接告訴你 A 距離鏡頭幾公分，除非另外用已知尺寸的物件或其他感測器做標定。用在避障、場景理解、背景虛化這類只需要相對遠近的任務很合適;用在尺寸量測或機械手臂定位就必須格外謹慎，這又是第 7 章「還原 vs. 腦補」的老問題。

有向邊界框(OBB，yolo26n-obb.pt)輸出的框可以旋轉，也就是多了一個角度參數。一般的邊界框永遠與影像的軸平行，遇到斜放的物件時，框裡會塞進大量背景(圖

16-14 量過這個「填充率」問題)。空拍影像中的船隻、停車場中斜停的車輛、輸送帶上隨機旋轉的零件、以及 PCB 上的元件，都是有向邊界框的主場。它同時也直接給出物件的方向，對機械手臂要決定夾爪角度這類任務非常實用——這比第 13 章用主軸或最小外接矩形去估方向省事得多。YOLO26 在這個任務上額外設計了角度損失，並處理了角度在 0 度與 180 度交界處的不連續問題(一個 179 度與一個 −179 度的框其實幾乎一樣，但單純相減會得到很大的誤差)。

最後是 YOLOE-26，一個開放詞彙(open-vocabulary)的變體。前面所有模型都只認得訓練時定義好的類別——COCO 的八十類，或你自己標註的那三類。開放詞彙模型則接受文字提示：你可以直接要求它找「戴安全帽的人」或「生鏽的螺絲」，即使訓練時沒有這些類別。它也接受視覺提示(給一張範例影像)，甚至可以無提示運作。對「類別很多、而且會一直增加」的場景(例如零售貨架上的上千種商品)，這是很有吸引力的方向。代價是準確度仍明顯低於專門訓練的模型，而且提示詞怎麼寫會顯著影響結果。務實的用法是把它當成「快速原型與資料標註的助手」：先用文字提示把可能的目標框出來，再由人工確認、最後仍然訓練一個專用模型上線。

把這幾個任務放在一起看，可以看出一條清楚的脈絡：YOLO 正在從「一個偵測器」變成「一組共用骨幹的視覺預測頭」。對工程實務的意義是，選任務的成本變低了——你可以先用偵測做出原型，發現需要面積就換 seg、需要方向就換 obb、需要遠近就換 depth，而資料管線、訓練指令與評估流程幾乎不用改。但有一件事沒有變便宜：標註。每換一種任務，標註的形式與成本就完全不同(圖 16-2 量過四種任務的輸出規模差了五個數量級)，而標註品質仍然是成績的天花板(圖 16-13)。所以選任務時真正該問的不是「模型支不支援」，而是「我付得起那個標註成本嗎」。

程式 16-5 分割、姿態估計與追蹤

```
from ultralytics import YOLO
import cv2

# --- 實例分割:輸出像素級遮罩 ---
seg_model = YOLO("yolo26n-seg.pt")
r = seg_model("parts.jpg")[0]
if r.masks is not None:
    print("偵測到物件數:", len(r.masks))
    mask = r.masks.data[0].cpu().numpy()       # 第一個物件的遮罩
    area_px = int(mask.sum())                  # 像素面積(可量測)
    print("面積(像素):", area_px)

# --- 姿態估計:輸出人體 17 個關鍵點 ---
pose_model = YOLO("yolo26n-pose.pt")
r = pose_model("person.jpg")[0]
if r.keypoints is not None:
    kpts = r.keypoints.xy[0]                   # 第一個人的關鍵點座標
    print("鼻子座標:", kpts[0].tolist())
    print("左肩座標:", kpts[5].tolist())

# --- 追蹤:第 12 章的偵測式追蹤 ---
cap = cv2.VideoCapture("traffic.mp4")
while True:
    ok, frame = cap.read()
```

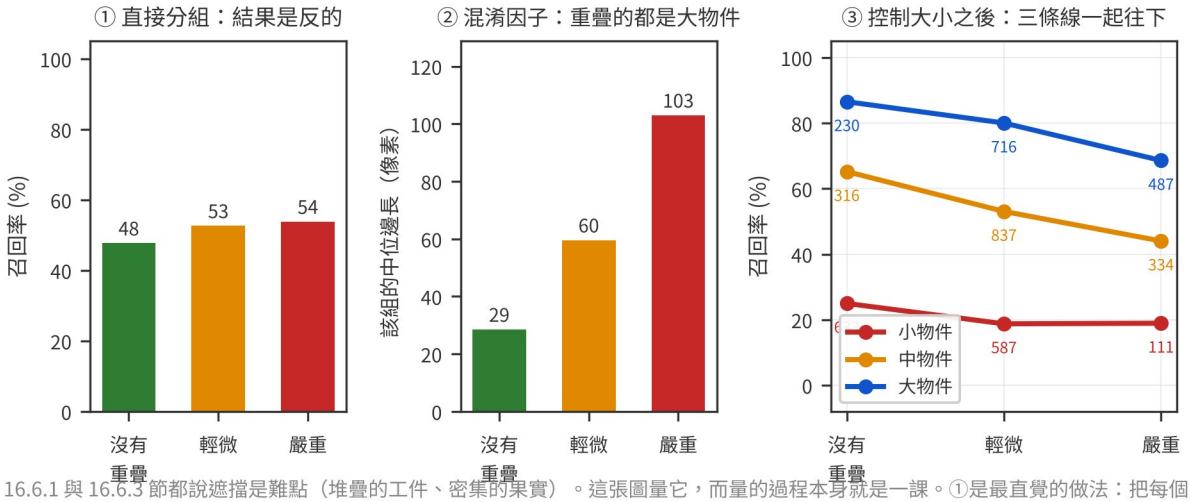
```
    if not ok:
        break
    # persist=True 讓 ID 跨影格維持
    res = seg_model.track(frame, persist=True,
tracker="bytetrack.yaml")[0]
    if res.boxes.id is not None:
        for box, tid in zip(res.boxes.xyxy, res.boxes.id):
            x1, y1, x2, y2 = box.tolist()
            cv2.putText(frame, f"ID {int(tid)}", (int(x1), int(y1)-5),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)
    cv2.imshow("track", res.plot())
    if cv2.waitKey(1) == 27:
        break
cap.release(); cv2.destroyAllWindows()
```

16.6 應用案例

16.6.1 案例一：AOI 瑕疵偵測

情境：產線上自動偵測工件表面的刮痕、裂紋、缺件。相較第 6 章的頻率域紋理檢測與第 9、10 章的傳統管線，YOLO 的優勢是能處理形態多變、難以用規則描述的瑕疵，且能同時分類多種瑕疵類型。流程：蒐集並標註各類瑕疵影像(通常需數百至數千張)→ 以遷移學習訓練 YOLO → 用 mAP 驗證 → 匯出為 ONNX 或 TensorRT 部署到產線電腦或 Jetson(第 17 章)。

關鍵挑戰是資料不平衡：產線上良品遠多於不良品，瑕疵樣本稀少。對策包括：資料增強(第 4、5 章)、合成瑕疵樣本、或改用只學習「正常」樣貌的異常偵測方法。這也解釋了為什麼第 6 章的傳統方法至今仍有價值——它不需要瑕疵樣本，對「沒見過的新瑕疵」天生有效，常與 YOLO 互補使用。



16.6.1 與 16.6.3 節都說遮擋是難點（堆疊的工件、密集的果實）。這張圖量它，而量的過程本身就是一課。①是最直覺的做法：把每個標註框按「它與同影像其他標註框的最大 IoU」分成四組，再看各組的召回率。結果卻是反的——完全不重疊的那組只有 48%，重疊嚴重的反而有 54%。難道遮擋反而有幫助?②揭穿了答案：這四組的物件大小差很多。不重疊那組的中位邊長只有 29 個像素，嚴重重疊那組卻有 103 個——相差 3.6 倍。道理其實很明白：大物件佔畫面大，自然容易跟別人重疊;而孤零零的小東西本來就不會跟誰重疊。於是①量到的根本不是遮擋的效果，是物件大小的效果（圖 16-4）。③是正確的做法：先把物件按大小分成三層，在每一層之內才比較重疊程度。這下三條線整齊地一起往下走——小物件從 25% 掉到 19%，中物件從 65% 掉到 44%，大物件從 87% 掉到 69%。遮擋確實有害，而且小物件被害得最慘（線上的小字是各組的框數）。這張圖最該帶走的不是那幾個百分比，而是那個方法：當你把資料分組來比較時，務必先問「這些組除了我關心的變數，還有什麼不一樣」。沒問這一句，①那種看起來很有說服力、方向卻完全相反的結論，隨時都會出現在你的報告裡。這是第 13、14 章一路強調的「別讓混淆因子跟著進來」，在偵測問題上的又一次現形。

圖 16-17　擁擠：遮擋吃掉多少召回率（以及一個混淆因子）

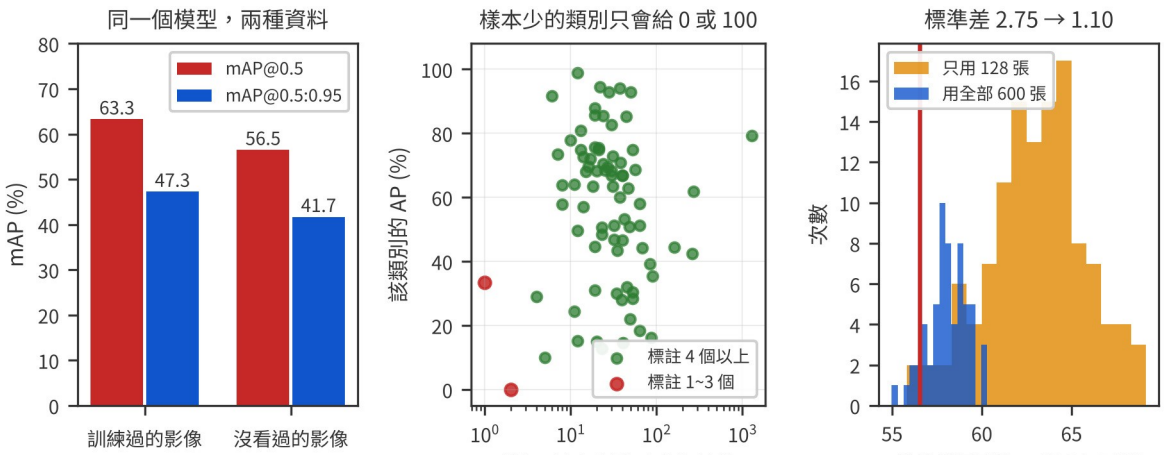
16.6.2 案例二：交通場景分析

情境：路口監控自動統計車流量、分類車種、偵測違規。這是第 12 章與本章的完整整合：YOLO 偵測並分類車輛與行人 → 內建追蹤器維持每個目標的 ID → 依軌跡越線計數(第 12.7.1 節)、結合第 4 章相機校正換算實際速度(第 12.7.2 節)、或判斷是否闖紅燈、逆向。這條管線串起了第 4、12、16 章，是智慧交通的標準架構。

16.6.3 案例三：農業與畜牧應用

情境：這是雲林在地產業最有潛力的應用之一。以無人機或固定攝影機搭配 YOLO，可自動計數果樹上的果實以估算產量、偵測病蟲害的葉片斑點、辨識雜草以做精準除草、或在畜牧場中計數與追蹤牲畜、透過姿態估計監控其行為異常(如久臥不起可能是生病徵兆)。

這類應用的工程重點在於：田間環境的光線變化劇烈(第 5、8 章的白平衡與對比強化成為必要的前處理)、物件常相互遮擋(密集果實的偵測是難點)、且需在無網路的田間即時執行(第 17 章的邊緣部署)。這也再次體現本書一貫的訊息：先進的 AI 模型不是孤立運作的，它需要前面各章的影像處理技術一起支撐，才能在真實世界中可靠工作。



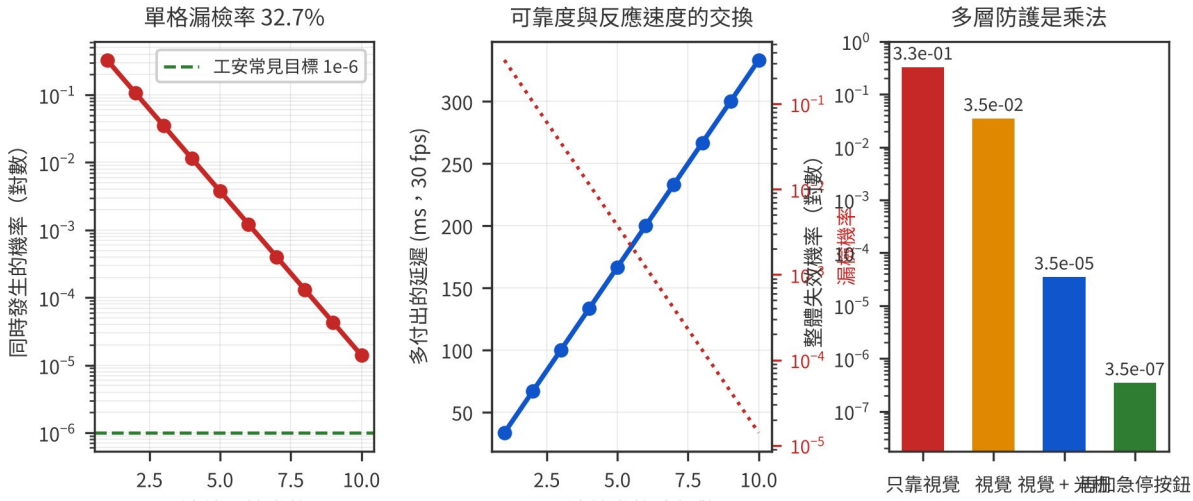
這張圖問兩個 16.3.3 節沒問的問題：你報的那個 mAP 是在什麼資料上量的？誤差範圍又有多大？左圖先回答第一個，而答案可能讓人不太舒服。同一個 YOLO26n，在 COCO128 上得到 mAP@0.5 63.3%，在 600 張沒看過的 val2017 影像上只有 56.5%，差了 6.8 個百分點；mAP@0.5:0.95 則是 47.3% 對 41.7%。原因很簡單：COCO128 是從 COCO 的「訓練集」裡抽出來的 128 張，模型早就看過它們。這不是 COCO128 的錯——它本來就是給人試跑流程用的小玩具；錯的是拿它當驗收成績。第 14 章講過「評估模型絕不能用它訓練時看過的資料」，這張圖就是那句話在偵測問題上的現場：一個 10 個百分點的落差，足以讓任何選型結論翻盤。中圖與右圖回答第二個問題。保留集上有標註的 79 個類別中，有 2 類的標註數只有 1 到 3 個，其中 1 類的 AP 剛好是 0% 或 100%——那不是能力，是樣本太少時 AP 只有幾種可能值。右圖量整體的不確定度：只用其中 128 張時，重抽樣的 mAP 標準差是 2.75 個百分點、95% 區間 57.4% 到 68.1%；用全部 600 張則降到 1.10——老規矩，誤差大致隨樣本數的平方根下降。三條守則：報 mAP 一定要說清楚測試集的來源與大小；比較模型要用同一份、而且模型都沒看過的資料；以及你真正在乎的那一類要有足夠樣本，否則它的 AP 只是在擲骰子。順帶一提，本章自寫的 mAP 實作在這份保留集上得到 56.52%，與 Ultralytics 官方工具的結果相差約一個百分點，差異來自群體標註與面積過濾等規則——跨工具比較數字時，先確認兩邊算的是同一件事。

圖 16-18　你報的那個 mAP：在哪種資料上量的，誤差又有多大

16.6.4 案例四：人機協作的安全監控

情境：在協作型機器人(cobot)的工作區，以 YOLO 的姿態估計即時追蹤作業員的手部與身體位置，當人員進入機器人的危險作業半徑時，自動減速或停機。這是工業安全的重要應用，也直接連結智慧機器人學程的核心關懷。

但這個案例也必須談責任：安全攸關的系統，不能單靠一個可能誤判的 AI 模型。工程上的正確做法是「多層防護」——視覺系統作為第一道預警，搭配實體光柵、壓力感測墊、緊急停止按鈕等獨立的安全機制。呼應叫獸開講與第 15 章談過的深度學習侷限(對抗脆弱、分布偏移)，在人命關天的場合，對 AI 保持審慎、設計冗餘，是工程師的基本責任。



16.6.4 節說人機協作系統不能單靠一個可能誤判的 AI 模型，要做多層防護。這張圖把那句話變成兩個數字。第一是整體失效機率：在保留集的 1284 個「人」的標註上，YOLO26n 以預設設定單格漏檢 420 個，漏檢率 32.7%。這個數字要公平地讀：COCO 影像裡的「人」有很多是遠處的小身影，而協作機器人工作區看到的是近距離的大目標，圖 16-17 量過大物件的召回率是 94%，對應的漏檢率約 6%。所以真實場域的單格漏檢率會比這裡低，但重點不變——它絕不是零，而機械手臂不會等你。但偵測是逐格進行的，所以第二步是連續多格的要求：若假設各格的漏檢彼此獨立，連續 3 格都漏的機率是 3.50e-02，連續 5 格是 3.74e-03。左圖畫的就是這條指數下降的線。代價在中圖：30 fps 下，要求連續 3 格等於多付出 100 毫秒的反應延遲，連續 5 格是 167 毫秒——而機械手臂在這段時間裡還在動。可靠度與反應速度是直接對換的。右圖是 16.6.4 節真正的重點：把獨立的防護層乘起來。視覺連續 3 格、加上實體光柵、再加上急停按鈕，整體失效機率可以壓到 3.5e-07。這裡有兩個必須講清楚的前提，否則這張圖會變成危險的樂觀。其一，「獨立」是關鍵假設，而它常常不成立——連續影格的漏檢高度相關（同一個遮擋會連漏很多格），所以真實的連續漏檢機率遠高於單格機率的三次方。其二，機率相乘只在「各層失效原因不同」時才成立;這正是為什麼安全層要用不同的物理原理（光、壓力、人手），而不是裝三台一樣的相機。這一段算術不是為了讓你放心，是為了讓你知道「我的模型準確率 95%」這句話，在安全系統裡什麼也沒保證。

圖 16-19　安全攸關系統：單一模型的失效率與多層防護

16.7 技術倫理：工程師的責任

本節不談程式，而是承接叫獸開講，把全書零散提及的倫理線索收攏起來——因為 YOLO 這樣強大而易用的工具，正是最需要談責任的地方。

回顧本書的倫理伏筆：第 7 章的「還原 vs. 腦補」提醒我們區分「找回的資訊」與「編造的資訊」，並在司法、醫療場合誠實標註;第 9 章的醫學影像分割強調自動結果僅供輔助、須由醫師確認;第 12 章的跌倒偵測提醒照護應用涉及被監控者的隱私與尊嚴;第 14 章的肺炎 X 光案例揭露資料偏誤如何讓模型學到無意義的捷徑;第 15 章列舉了深度學習的黑箱與對抗脆弱。這些線索指向同一個核心：技術能力愈強，對後果的想像力就愈重要。

物件偵測尤其敏感，因為它的用途橫跨光譜的兩端：同一套人流分析技術，可以用於商場最佳化動線、也可以用於監控特定族群;同一套目標追蹤技術，可以用於搜救失蹤者、也可以用於軍事鎖定。Redmon 的選擇是退出，但那不是唯一的答案。工程師還有許多實際的作為：在系統設計中內建隱私保護(如只統計數量不儲存人臉、在裝置端處理而非上傳雲端)、對資料來源與模型限制保持透明、在授權條款中明訂禁止的用途、在部署前評估誤判的後果與受影響者、以及——最基本卻最重要的——在被要求做一件你認為不對的事情時，有能力說出自己的疑慮。

叫獸想說的是：這門課教你把技術做出來，但沒有任何一行程式碼能替你決定「該不該做」。那是你作為一個專業工作者、也作為一個人的判斷。而判斷力，和寫

程式一樣，是需要練習的——從現在開始，每當你部署一個系統，多問自己一句：「誰會受益?誰可能受害?如果它出錯了，誰要承擔?」

16.7.4 推導：把「安全」與「隱私」寫成可以驗收的數字

16.6.4 與 16.7 節談了多層防護與隱私保護，但倫理若停在態度層次，就只是漂亮話。這一小節把那兩件事變成可以寫進規格書、也可以被稽核的算式。這不是要用數學取代判斷，而是要讓判斷有東西可以討論——一個說不出失效率的安全承諾，和沒有承諾是一樣的。

先看安全。設偵測器在單一影格上漏掉目標的機率是 q。若要求「連續 k 格都偵測不到才算真的沒看到」，且假設各格的漏檢彼此獨立，則整段判斷失效的機率是 q 的 k 次方。圖 16-19 以保留集的 1,284 個「人」實測 q ≈ 0.33(其中包含大量遠處的小身影;若只看近距離、不被遮擋的大目標，依圖 16-17 的量測 q 約為 0.13)，於是連續 3 格全漏的機率約 0.035、連續 5 格約 0.0037。代價是延遲：在 30 fps 下，多等 k 格就是多 k/30 秒，k = 3 是 100 毫秒、k = 5 是 167 毫秒——而機械手臂在這 167 毫秒裡還在移動。所以 k 不是愈大愈好，它由「可容忍的反應距離」反推：若手臂速度是 v、安全距離是 d，則 k/30 × v 必須遠小於 d。

接著是多層防護的算術。若有若干道彼此獨立的防護，整體失效機率是各層失效機率的乘積。視覺連續 3 格(0.035)、實體光柵(約 10^{-3})、急停按鈕(約 10^{-2})串起來，整體約 3.5×10^{-7}，這才接近工安領域常見的目標值。

但這裡有兩個前提必須說破，否則上面那串乘法會變成危險的樂觀。其一，「獨立」幾乎從不成立。連續影格的漏檢高度相關——同一個遮擋、同一個逆光、同一個被 NMS 刪掉的框(圖 16-9)，會讓好幾格一起漏;真實的連續漏檢機率因此遠高於 q 的 k 次方。工程上的對策是把 q 的估計做在「最壞情境」而非平均情境上：量的是逆光遮擋、極端姿態時的漏檢率。其二，各層必須用不同的物理原理才算獨立。裝三台一模一樣的相機不會讓失效率變成三次方，因為讓第一台失效的霧氣、逆光或污損，會同時讓另外兩台失效。光、壓力、人手——原理不同，才是真正的獨立。

再看隱私。去識別化本身也是一個偵測任務，因此它也有召回率，而「漏掉一張臉」就是一次直接的隱私失效。圖 16-20 實測了這件事：一張有四個人的照片，姿態模型只圍出三個頭部，另一個因為背對鏡頭、臉部關鍵點信心不足而漏掉。所以這類管線的驗收標準不是準確率，而是「最壞情況下有沒有漏」。可操作的設計有三條：把臉部偵測的門檻調到極低，寧可多模糊(此處誤報幾乎沒有代價);偵測不到臉時，直接模糊整個人體框作為保底;以及最根本的一條——在管線的最前端就不保存原始影像，只留下框的座標與時間戳。



16.7 節把「內建隱私保護」列為工程師的實際作為之一，但沒說怎麼做、代價多少。這張圖做一次給你看。做法是把去識別化直接接在偵測管線後面：先用姿態模型的臉部關鍵點（鼻、雙眼、雙耳）圍出頭部範圍，再對那一塊做高斯模糊（第 5 章）。結果是中間那張：3 個頭被遮蔽，只佔畫面的 0.7%，而偵測器之後仍然數得出 4 個人（原圖 4 個）、姿態模型仍然抓得到 39 個可信關鍵點（原圖 41 個）——也就是說，人流計數、區域進出、安全距離、跌倒偵測這些功能完全不受影響。右邊那張是粗暴的做法：把整個人都模糊掉，遮蔽 27.4% 的畫面，偵測到的人數掉到 1 個、可信關鍵點只剩 12 個——功能跟著一起消失。這裡要補一個實測得到的細節：模糊的「強度」也會改變結果。同樣是全身模糊，核小一點時偵測器照樣數得出四個人，核大到足以抹掉輪廓才會失效。換句話說，弱的模糊既保護不了身分、也沒省下什麼，是兩頭落空的設定——這種參數必須實測，不能憑感覺調。這張圖想說的是：隱私保護不是「要或不要」的二選一，而是一個可以設計的取捨。多數監控類應用真正需要的是「有幾個人、在哪裡、往哪走」，而不是「這個人是誰」;把身分資訊在管線的最前端就抹掉，功能幾乎不損失。更進一步的做法是根本不存原始影像——只存框的座標與時間戳，影像處理完就丟。這些都是可以寫進規格書、也可以被稽核的工程決定，比事後宣稱「我們很重視隱私」有意義得多。最後呼應 16.7 節那句話：沒有任何一行程式碼能替你決定該不該做，但一旦決定要做，怎麼做得更負責任，是有技術答案的。

圖 16-20　去識別化：把隱私保護做進管線裡

最後把這兩件事收攏成一個共同的形狀。安全與隱私的工程處理，都是把一個模糊的承諾(「我們很重視」)換成三樣具體的東西：一個可以量的指標、一個明確的最壞情境、以及一個在模型失效時仍然有效的保底機制。第 14 章說機器學習真正難的不是訓練模型，是誠實地知道它有多好;本章要補的下半句是——知道它有多好之後，你還得決定，這樣夠不夠好到可以上線。那個決定沒有公式，但它應該建立在有公式的那一部分之上。

16.8 本章與全書的觀念連結總表

表 16-2 第 16 章與全書其他章節的觀念連結

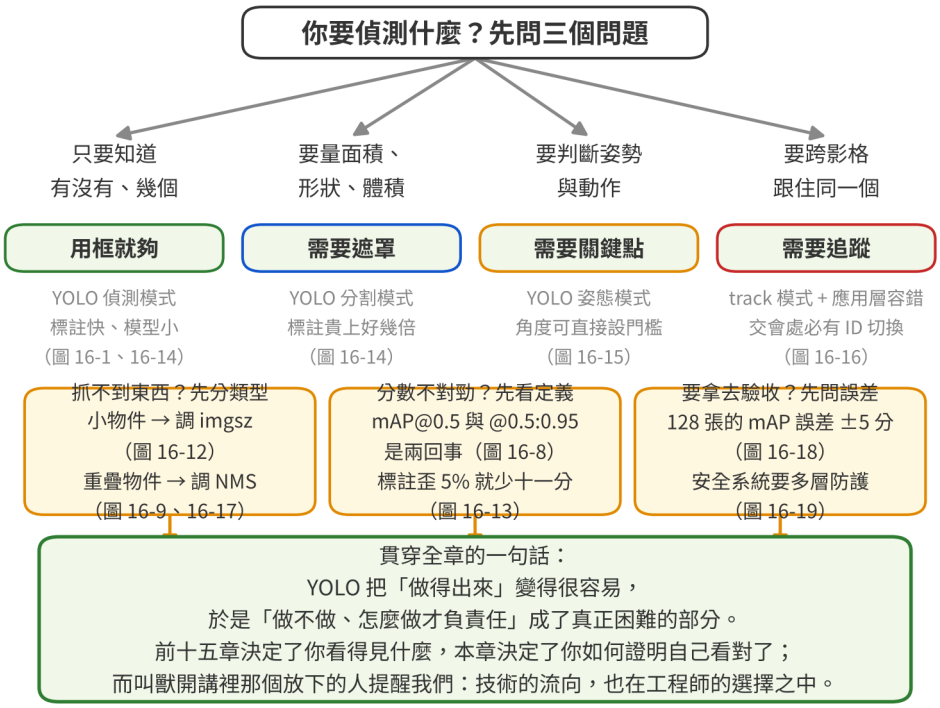


圖 16-21　第 16 章的決策地圖

程式實作 Lab 16

Lab 16-1 YOLO 快速上手

目標：熟悉推論與參數。(a)安裝 Ultralytics，以程式 16-2 對數張照片(街景、室內、多物件場景)推論，觀察 COCO 八十類的偵測結果;(a2)同一張照片改用 nms=False 再跑一次，比較兩個預測頭的框數、信心分數與耗時，並說明免 NMS 的輸出為什麼固定是 300 個框;(b)調整 conf 門檻(0.1、0.25、0.5、0.8)，記錄漏檢與誤報如何變化，對照第 14 章的精確率-召回率權衡;(c)調整 iou 門檻，觀察 NMS 如何影響重疊框的保留;(d)比較 yolo26n 與 yolo26s/m 的偵測結果與推論速度(FPS)，體會精度與速度的取捨;(e)討論：在什麼應用中你會把 conf 調低?什麼應用中會調高?

Lab 16-2 訓練自訂偵測器

目標：完成一個屬於自己的偵測器。(a)選定二到三類物件(如實驗室器材、特定零件、水果)，各拍攝並標註五十張以上影像(建議用 LabelImg 或 Roboflow);(b)依第 14 章原則切分訓練/驗證集，撰寫 data.yaml;(c)以程式 16-4 從預訓練權重開始訓練(遷移學習)，觀察訓練過程的損失曲線與 mAP 變化;(d)在驗證集上計算 mAP@0.5 與 mAP@0.5:0.95，並找出模型最常誤判的情況;(e)刻意減少訓練資料到二十張重訓，比較 mAP 差異——驗證第 14 章「資料是燃料」;(f)加入或移除資料增強參數，觀察對過度擬合的影響。

Lab 16-3 姿態估計與跌倒偵測

目標：實作第 12 章預告的跌倒偵測。(a)以程式 16-5 的 pose 模型對人物影片做姿態估計，視覺化十七個關鍵點與骨架;(b)計算人體重心(可用髖部關鍵點近似)的高度

與其變化速率;(c)設計一個簡單的跌倒判斷規則：重心高度在短時間內急遽下降、且之後維持在低位;(d)測試：正常坐下、蹲下、躺下與真正的跌倒，你的規則能否區分?(e)討論：承第 12.7.3 節，這類照護系統的誤報與漏報各有什麼代價?如何在系統設計中兼顧隱私(例如只傳送骨架資料而非影像)?

Lab 16-4 交通流量分析系統

目標：整合第 12 章與本章，完成完整的視訊分析系統。(a)以 YOLO 的 track 模式對交通影片偵測並追蹤車輛，確認 ID 跨影格穩定;(b)畫一條虛擬計數線，依軌跡越線方向統計雙向車流量;(c)分類統計不同車種(汽車、機車、公車、卡車)的數量;(d)進階：結合第 4 章相機校正，把像素位移換算成實際速度(第 12.7.2 節);(e)量測系統的 FPS，評估在你的硬體上能否即時處理，並思考第 17 章的部署選項(用更小的模型?降低解析度?用 GPU?)。

Lab 16-5 自己寫一次 IoU、NMS 與 mAP

目標：把 16.3 節的三個概念從「看得懂」變成「寫得出來」，並親手驗證自己的實作與官方工具一致。(a)寫一個 iou(a, b) 函式，並用三組手算的例子驗證：完全重合（應為 1）、完全不相交（應為 0）、以及一個 100×100 的框水平偏 33 像素（應約為 0.5）;特別檢查你的程式在不相交時有沒有算出負面積。(b)寫一個 nms(boxes, scores, thr) 函式，對同一張影像先以 iou=1.0 取得未經抑制的原始輸出，再用自己的 NMS 處理，確認結果與 Ultralytics 內建的一致;並畫出「門檻 vs 保留框數」的曲線（對照圖 16-6）。(c)寫一個配對函式：預測依分數由高到低，每個預測找 IoU 最大且尚未被認領的同類真實框，超過門檻才算命中、每個真實框最多認領一次;輸出 TP 標記陣列。(d)由 TP 標記累積出精確率-召回率曲線，實作全點內插 AP 與十一點內插 AP，比較兩者的差距。(e)對整個資料集逐類別計算 AP 再平均得到 mAP@0.5，與 model.val() 的輸出比較，若有差距請找出原因（提示：群體標註、面積過濾、以及低於信心門檻的預測有沒有被算進去）。(f)把 IoU 門檻從 0.50 掃到 0.95 計算十個 mAP 並平均，確認你算得出 mAP@0.5:0.95。延伸：把 conf 從 0.001 改成 0.25 重跑一次 (e)，觀察 mAP 被低估多少，並解釋原因。

Lab 16-6 把一個偵測器誠實地驗收一次

目標：訓練一個自己的偵測器，然後用本章的工具把它的能力與極限量清楚——重點不在分數多高，而在你能說出這個分數可以信幾分。(a)選定一到三類物件，拍攝並標註;切分時務必按「來源」分組（依拍攝日期、批次或場景），不要隨機打散，並在報告中說明你的切分依據。(b)以程式 16-4 從預訓練權重開始訓練，記錄訓練與驗證的 mAP 曲線，確認及早停止有沒有生效。(c)依物件面積分成小、中、大三組，分別計算召回率（對照圖 16-4）;再依「與鄰居的最大 IoU」分組，但必須在每個大小層之內比較（對照圖 16-17，想想為什麼不能直接分組）。(d)掃描信心門檻與 NMS 門檻各十個值，畫出精確率、召回率與 F1 的曲線，依你的應用代價（漏抓貴還是誤報貴）選定一組門檻並說明理由。(e)用重抽樣估計 mAP 的 95% 區間：對測試影像有放回地抽樣一百次、每次重算 mAP，取 2.5% 與 97.5% 分位數;據此說明你的模型與一個基準模型是否真的有差。(f)把人工標註隨機抖動 5% 再重算一次成績，用這個數字說明「標註品質的天花板」在哪裡（對照圖 16-13）。(g)寫一份一頁的驗收報告，內容必

須包含：測試集的影像數與標註框數、mAP 及其誤差範圍、逐類別與逐尺寸的召回率、選定的門檻與理由、以及一節「已知失效情境」（列出至少三種你實際觀察到模型會失敗的狀況）。延伸：若你的應用屬於安全攸關類型，依 16.7.4 節的算式估計單格漏檢率與連續多格的失效機率，並設計一套不依賴視覺的保底機制。

本章重點整理

- 四種視覺任務依精細度遞增：分類(是什麼)、偵測(是什麼加在哪裡)、語意分割(每像素類別、不分個體)、實例分割(每像素類別且分個體);姿態估計則輸出關鍵點骨架。
- 兩階段偵測器先提候選再分類(準但慢)，單階段偵測器一次前向傳播直接輸出所有結果(快);YOLO 把偵測重新定義為迴歸問題，只看一次全圖，並能利用全域脈絡減少背景誤判。
- 關鍵概念：IoU(交集聯集比，判定框準不準)、NMS(保留信心最高框、抑制重疊者，與第 9 章 Canny 同名同理)、mAP(建立於第 14 章精確率與召回率，常用 mAP@0.5 與 mAP@0.5:0.95)。
- Ultralytics YOLO26 提供 n/s/m/l/x 多種尺寸供依算力選擇，並同時訓練一對多(需 NMS)與一對一(免 NMS)兩個預測頭，以 nms=False 切換;它移除了 DFL、改用漸進式損失與小目標感知標籤指派，並擴充出語意分割、單眼深度估計、有向邊界框等任務;訓練自訂資料集三步驟：標註資料(YOLO 格式)、撰寫 data.yaml、train 與 val;訓練參數中的旋轉、亮度、翻轉即第 4、5 章的資料增強，從預訓練權重開始即第 15 章的遷移學習。
- 延伸功能：seg 模型輸出像素級遮罩(可精確量測面積)、pose 模型輸出十七個人體關鍵點(動作分析、跌倒偵測、人機協作安全)、track 模式實現第 12 章的偵測式追蹤並維持穩定 ID。
- 技術倫理：同一套技術可用於截然不同的目的;工程師的實際作為包括內建隱私保護、對限制保持透明、授權中明訂禁止用途、評估誤判後果、以及在必要時表達疑慮。安全攸關系統須設計多層防護，不可單靠可能誤判的 AI 模型。

習題

1. 請區分影像分類、物件偵測、語意分割與實例分割，說明各自的輸出形式與難度差異。

    提示與參考解答：分類：輸出整張影像的類別(是什麼)，不含位置。偵測：輸出每個物件的類別與邊界框(是什麼加在哪裡)。語意分割：每個像素標上類別，有精確輪廓但不分同類個體(三個人是同一片「人」)。實例分割：每像素類別且區分不同個體(三個人各有獨立輪廓與編號)，最精細也最困難。

2. 兩階段與單階段偵測器有何差異?YOLO 的「只看一次」具體是什麼意思?

    提示與參考解答：兩階段(如 R-CNN 系列)先產生候選區域再逐一分類與微調框，準確度高但慢(同一影像被網路處理多次)。單階段(YOLO、SSD)一次前向傳播就同時輸出所有物件的位置與類別。YOLO 把影像切成網格，讓網路對每個網格直接預測有無物件、框的位置、類別與信心，整張影像只需通過網路一次，故名 You Only Look Once。

3. YOLO 每次都看整張影像，這帶來什麼額外好處?它早期的弱點是什麼?

   提示與參考解答：好處：能利用全域脈絡資訊(如「周圍都是道路，這個模糊物體較可能是車而非船」)，背景誤判較少。早期弱點：對小物件與密集群體較不擅長;YOLOv3 引入多尺度預測(呼應第 11 章多解析度)後大幅改善。

4. 什麼是 IoU?它如何用於判定偵測是否正確?

   提示與參考解答：IoU(Intersection over Union)是預測框與真實框的交集面積除以聯集面積。IoU 為 1 代表完美重合、0 代表完全不重疊。實務上以 IoU 超過某門檻(常用 0.5)判定該偵測為正確;也用於 NMS 判斷兩框是否重複。

5. 說明 NMS 的運作步驟。它與第 9 章的哪個步驟同名同理?

   提示與參考解答：步驟：取信心分數最高的框，刪除與它 IoU 超過門檻的其他框(視為重複)，對剩下的框重複此程序直到處理完畢。它與第 9 章 Canny 邊緣偵測第三步的「非極大值抑制」同名同理——都是「保留局部最強、抑制鄰近重複」，體現本書「核心思想以不同面貌反覆出現」的主軸。

6. mAP 是如何計算的?它建立在第 14 章的哪兩個指標之上?mAP@0.5 與 mAP@0.5:0.95 有何差別?

   提示與參考解答：對每個類別，調整信心門檻得到精確率-召回率曲線，曲線下面積為該類的 AP;所有類別的 AP 取平均即 mAP。它建立於第 14 章的精確率與召回率。mAP@0.5 以 IoU 門檻 0.5 判定正確;mAP@0.5:0.95 在多個 IoU 門檻(0.5 到 0.95)下取平均，標準更嚴格，是 COCO 的主要指標。

7. 訓練自訂 YOLO 模型的三個步驟是什麼?YOLO 標註格式的每一行記錄什麼?

   提示與參考解答：三步驟：一、準備並標註資料(用 LabelImg/Roboflow 等，依第 14 章切分訓練/驗證/測試集);二、撰寫 data.yaml 設定檔指定路徑與類別名稱;三、呼叫 train() 訓練、val() 計算 mAP。標註格式每行為「類別編號 中心 x 中心 y 寬 高」，座標皆正規化到 0 至 1。

8. 程式 16-4 的訓練參數 degrees、hsv_v、fliplr、patience 分別對應本書哪幾章的觀念?「從預訓練權重開始訓練」又是什麼?

   提示與參考解答：degrees(旋轉)、fliplr(水平翻轉)是第 4 章幾何轉換;hsv_v(亮度)是第 5、8 章;三者皆作為資料增強對抗過度擬合(第 14 章)。patience 是第 14 章的及早停止。從預訓練權重開始訓練即第 15 章的遷移學習——借用已學好的通用特徵，少量資料也能訓練出高效能模型。

9. YOLO 的 seg、pose、track 三種模式各輸出什麼?各舉一個應用。

   提示與參考解答：seg：輸出像素級遮罩(實例分割)，應用如精確量測不規則物件面積、背景去除。pose：輸出人體十七個關鍵點骨架，應用如動作分析、跌倒偵測、人機協作安全監控。track：每一影格偵測並跨影格關聯維持穩定 ID(第 12 章偵測式追蹤)，應用如人流計數、車流分析、軌跡記錄。

10. AOI 瑕疵偵測用 YOLO 會遇到什麼資料上的挑戰?為什麼第 6 章的傳統方法至今仍有價值?

    提示與參考解答：挑戰是資料不平衡：產線上良品遠多於不良品，瑕疵樣本稀少，難以訓練。對策：資料增強、合成瑕疵樣本、或改用只學正常樣貌的異常偵測。第 6 章的頻率域紋理檢測不需要瑕疵樣本(策略是「濾掉正常」)，對「沒見過的新瑕疵」天生有效，故與 YOLO 互補使用。

11. 為什麼人機協作的安全監控不能單靠 YOLO 姿態估計?應如何設計?

提示與參考解答：因為深度學習模型可能誤判(第15 章：對抗脆弱、分布偏移、黑箱難解)，而人機協作安全攸關人命。正確做法是多層防護：視覺系統作為第一道預警，搭配實體光柵、壓力感測墊、緊急停止按鈕等獨立的安全機制，形成冗餘。對AI 保持審慎、設計冗餘，是工程師的基本責任。

12. (思辨題)承叫獸開講：Redmon 因軍事應用與隱私疑慮退出電腦視覺研究。請說明支持與反對的論點，並提出「除了退出之外」工程師還能有哪些負責任的作為。

    提示與參考解答：本題無標準答案，評分重點在論證品質。反對方：技術本身中性，道德在於使用者;你不做別人也會做，退出反而失去影響力。支持方：創造者對創造物的流向並非全然無責;當下行風險遠大於上行效益時，不推進本身即是負責任的表態。其他作為：系統設計內建隱私保護(只統計數量不存人臉、端上處理不上傳)、對資料來源與模型限制保持透明、授權條款明訂禁止用途、部署前評估誤判後果與受影響者、以及在被要求做不對的事時有能力表達疑慮。

13. 為什麼說「只看一次」真正省下來的是重複的特徵萃取?請用滑動視窗的成本估算說明，並指出現代兩階段偵測器與樸素滑動視窗的差別。

    提示與參考解答：樸素滑動視窗要對每一個候選位置各跑一次分類器，而相鄰視窗大量重疊，等於把同一塊影像的特徵重複算了上萬遍。圖16-3 實測：對一張810×1080 的照片用三種框、步長16 掃過，要分類7,408 次;以實測的每次7.4 毫秒估算，共需約55 秒。同一台機器上YOLO26n 看完整張圖只要約55 毫秒，差了約1,000 倍。YOLO 整張圖只萃取一次特徵，再讓8,400 個預測點共用這份特徵（圖16-2）。要公平的是：Faster R-CNN 之後的兩階段偵測器同樣只萃取一次特徵，只是多了「提候選區域」這一步，因此它比單階段慢的是倍數而非數量級;圖16-3 量的是最樸素的做法，用來凸顯重複計算的代價。

14. YOLO26 的一對多預測頭在 640×640 的輸入下會輸出一個 84×8400 的張量，而一對一預測頭輸出的是 (N, 300, 6)。請說明這兩個數字各自的來源，以及它與「多尺度預測」的關係。

    提示與參考解答：84 = 4 + 80，即每個預測點輸出4 個框座標與80 個COCO 類別分數（採用無錨框設計，不再有物件存在與否的獨立分數）。8400 = 80×80 + 40×40 + 20×20 = 6400 + 1600 + 400，對應三個解析度的特徵圖，格子邊長（stride）分別是8、16、32 個像素。細的格子多、負責小物件，粗的格子大、負責大物件，這就是16.2.3 節說v3 引入多尺度預測的實體，也是第11 章影像金字塔思想的延續。可以自己驗證：640/8 = 80、640/16 = 40、640/32 = 20，這也說明imgsz 為什麼必須是32 的倍數。

15. IoU 0.5 到底有多鬆?請以一個 100×100 的框為例，算出 IoU 等於 0.5 時容許的水平偏移量，以及容許的邊長倍率範圍。這對「mAP@0.5 很高」這句話有什麼啟示?

    提示與參考解答：水平平移d 時交集是(100−d)×100、聯集是(100+d)×100，故IoU = (100−d)/(100+d);令它等於0.5 得d = 100/3 ≈ 33，即偏掉三分之一個身位仍算「正確偵測」（圖16-5 實測33.5%）。同中心、邊長比s > 1 時，交集是小框、聯集是大框，IoU = 1/s²;令它等於0.5 得s = √2 ≈ 1.41，縮小側同理是1/√2 ≈ 0.71。啟示：mAP@0.5 是一個相當寬鬆的標準，它問的是「有沒有找到東西」;要衡量「框畫得準不準」必須看mAP@0.5:0.95。圖16-8 實測同一個模型兩者分別是56.5% 與41.7%，差了14.8 個百分點。若應用需要精確位置（機械手臂抓取、尺寸量測），IoU 0.5 的評分根本不能用。

16. NMS 的 IoU 門檻與評分用的 IoU 門檻有什麼不同?若把 NMS 門檻從 0.7 調到 0.3，精確率與召回率會怎麼變?為什麼?

    提示與參考解答：兩者只是剛好同名。NMS 的門檻問「這兩個預測是不是在講同一個物件」，用於後處理去重;評分的門檻問「這個預測跟標準答案夠不夠接近」，用於計算 TP/FP。NMS 門檻調低會刪掉更多框：輸出變少，誤報跟著減少（精確率上升），但真的疊在一起的兩個物件也會被刪掉一個（召回率下降）。圖 16-9 實測：門檻從 0.2 調到 0.95，召回率上升而精確率大幅下降，F1 在 0.5 附近最高，但那個最佳值完全取決於場景有多擠。該怎麼調取決於場景：稀疏場景（產線一次一件）可以調低;密集場景（人群計數、果實計數）必須調高並接受誤報變多。

17. NMS 有一個無解的兩難，是什麼?請舉一個實際場景，並說明有哪些對策。

    提示與參考解答：NMS 分不清「同一個物件的重複框」與「兩個真的疊在一起的物件」。圖 16-9 的實例：保留集裡兩個同類物件的真實框彼此 IoU 就有 0.66，只要 NMS 門檻設在這個值以下，其中一隻必定被當成重複刪掉——而且是在偵測器正確找到它之後才刪的。實際場景：人群計數、貨架補貨、密集果實採收、停車場車位偵測。對策：（1）提高 NMS 門檻並以其他方式過濾誤報;（2）改用 Soft-NMS（把重疊框的分數降低而非直接刪除）;（3）改用不需要 NMS 的架構（DETR 系列以集合預測與匈牙利配對取代後處理）;（4）在應用層容錯，例如只在固定計數線上判斷。

18. mAP 的 m 是什麼意思?為什麼「mAP 很高」不保證你在乎的那一類表現好?請以本章實測說明。

    提示與參考解答：m 是 mean，指「對所有類別的 AP 取平均」。平均會藏住個別類別的災難。圖 16-7 實測：保留集上有標註的 79 個類別，AP 從 100%（airplane、train 等）到 0%（scissors、mouse）都有，平均起來是 56.5%;其中 person 的 AP 是 0.792、而最差的幾類是 0。如果你的應用真正在乎的剛好是墊底的那一類，那個 56.5% 完全沒有意義。正確做法是看逐類別的 AP 與混淆情形，這與第 14 章「看混淆矩陣而不是看平均」是同一課。另外要注意樣本數：圖 16-18 顯示標註不到四個的類別，AP 多半不是 0 就是 100，那是雜訊而非能力。

19. 為什麼計算 mAP 時信心門檻必須設到極低（例如 0.001），而實際部署時卻該用 0.25 以上?把兩者搞混會發生什麼事?

    提示與參考解答：AP 是精確率-召回率曲線下的面積，而曲線的高召回率端必須由低分預測提供。若先用 0.25 砍掉低分預測，曲線右端就永遠補不回來，AP 會被系統性低估。部署時則相反：低門檻會產生大量誤報，圖 16-11 實測門檻 0.02 時輸出 14,469 個框，而真實物件只有 4,251 個，超過七成是誤報。搞混的後果有兩種：用高門檻算 mAP 會低估自己的模型、並在與別人比較時吃虧;用低門檻部署則會讓下游被垃圾框淹沒。實務建議：驗證腳本與部署設定分開寫，並在程式碼裡註明理由。

20. 提高輸入解析度 imgsz 是對付小物件最有效的手段之一，但本章實測它在某個點之後反而讓總成績變差。請說明這個現象與它的成因，並提出正確的做法。

    提示與參考解答：圖 16-12 實測，imgsz 從 320 加到 1280 時，小物件召回率從 5% 一路升到 36%，但大物件召回率在 640 時最高（77%）、到 1280 掉到 70%;總成績因此呈倒 U 形，mAP@0.5 在 960 達到最高的 58.9%，1280 時反而掉回 53.9%，而耗時卻增加了好幾倍。成因是分布偏移（第 15 章）：預訓練權重是在 640 上訓練的，把輸入放大一倍，所有物件在網路眼中都大了一倍，大物件因此被推出了它熟悉的尺度範圍。正確做法：（1）推論的 imgsz 不要離訓練值太遠;（2）真的需要高解析度，就用高解析度重新訓練，而不是只在推論時放大;（3）把大圖切塊、各自以訓練解析度偵測再合併座標。

21. 標註品質對成績的影響有多大?為什麼 mAP@0.5 與 mAP@0.5:0.95 對標註誤差的敏感度差這麼多?

    提示與參考解答：圖16-13 的實驗把模型完全固定，只對人工標註加入隨機平移與縮放。抖動5%（肉眼幾乎看不出）時，mAP@0.5 從56.5% 只掉到55.8%，但 mAP@0.5:0.95 從41.7% 掉到31.4%，少了10.3 個百分點;抖動20% 時兩者分別掉到23.3% 與5.7%。敏感度差異的原因：mAP@0.5 只要求IoU 超過0.5，而由圖16-5，偏5% 的框IoU 仍有0.9 以上，輕鬆過關;mAP@0.5:0.95 本來就在量「框準不準」，標註歪了，準的定義就跟著歪。三個實務結論：標註品質是成績的天花板;比較模型必須用同一份標註;驗收標準要跟著標註精度走，標註一致性只有IoU 0.9 時就不該拿mAP@0.95 當KPI。

22. 在 128 張影像上量到 mAP@0.5 = 56.5%。這個數字的誤差範圍大約多少?如果另一個模型量到 59%，你能說它比較好嗎?

    提示與參考解答：圖16-18 以重抽樣（bootstrap）量得標準差約2.8 個百分點、95% 區間約51% 到62%。59% 完全落在這個區間內，因此不能說它比較好——差距沒有超過雜訊。要做出可信的宣稱有三條路：把測試集加大（圖16-18 實測128 張時標準差2.8、600 張時1.1，大致隨樣本數的平方根下降）;用交叉驗證或多次重抽樣並報告區間;或改用配對比較（兩個模型在同一批影像上逐張比較），這樣可以消去影像難度造成的共同變異，檢定力高得多。另外提醒：報mAP 一定要附測試集的影像數與標註框數。

23. (推導題)承 16.7.4 節：某偵測器對作業員的單格漏檢率是 6%。若要求連續 k 格都偵測不到才判定「沒有人」，請算出 k = 1、3、5 時的失效機率與 30 fps 下的延遲，並說明「各格獨立」這個假設為什麼通常不成立。

    提示與參考解答：假設獨立時失效機率是0.06 的k 次方：k = 1 為6×10^{-2}、k = 3 為2.16×10^{-4}、k = 5 為7.78×10^{-7}。30 fps 下延遲分別是33、100、167 毫秒。「獨立」通常不成立，因為造成漏檢的原因多半是持續性的：同一個遮擋、同一個逆光、同一個被NMS 刪掉的框（圖16-9）會讓連續好幾格一起漏;因此真實的連續漏檢機率遠高於單格機率的k 次方。工程對策：把漏檢率估計做在最壞情境（逆光、遮擋、極端姿態）而非平均情境;並且各防護層要用不同的物理原理（光柵、壓力墊、急停按鈕），裝三台相同的相機並不會讓失效率變成三次方。

24. (推導題)請說明全點內插 AP 的「單調化」步驟在做什麼、為什麼要做，並說明它與 2007 年 VOC 十一點內插法的差別。

    提示與參考解答：單調化是把每一點的精確率換成它右邊所有點的最大值，即$P^*(R) = \max\{P(R') : R' \geq R\}$，讓曲線變成單調不升的階梯，再積分面積。理由是：精確率曲線本身是鋸齒狀的（每多接受一個預測，精確率可能升也可能降），而既然實務上你總能選一個更好的門檻，曲線上的凹陷就不該算進成績。十一點內插法只在召回率0、0.1、…、1.0 這十一個點取（單調化後的）精確率再平均，是一種粗糙的取樣;全點內插則把每一段召回率的變化都算進去，是目前COCO 採用的方式。兩者用同一條曲線可以差好幾個百分點——圖16-7 實測person 類別：全點0.782、十一點0.758。所以比較論文數字時務必確認算法一致。

25. (綜合實作思辨題)你要為一座雲林的文旦果園做產量估算系統：以無人機拍攝、用 YOLO 計數樹上的果實。請規劃完整流程，指出這個任務最可能遇到的四個技術難題與各自對策，並說明你會如何向果農報告「這套系統有多準」。

    提示與參考解答：流程應涵蓋資料（拍攝規劃、按樹/按日期分組切分）、標註（框或遮罩、標註規範與一致性檢查）、訓練（從預訓練權重遷移學習、資料增強用第4、5 章技術）、評估（逐類別召回率、計數誤差）、部署（第17 章邊緣裝置）。

四個難題與對策：（1）小物件——遠距離拍攝的果實往往不到 32 像素，圖 16-4 實測這個尺寸的召回率只有兩成;對策是降低飛行高度、提高 imgsz（但注意圖 16-12 的倒 U）、或切塊偵測。（2）遮擋與密集——果實彼此重疊且被葉片遮擋，圖 16-17 實測控制大小後重疊愈嚴重召回率愈低;對策是提高 NMS 門檻、多角度拍攝、並接受「可見果實數」與「實際果實數」之間需要一個校正係數。（3）資料洩漏——同一棵樹、同一天的相鄰影像極度相似，隨機切分會讓驗證分數虛高;對策是按樹、按日期分組切分（GroupKFold）。（4）分布偏移——光線、季節、品種、生長階段都會變，訓練時的晴天中午資料無法代表陰天傍晚;對策是跨時段跨天候收集驗收資料，並規劃上線後的持續抽檢與重訓。報告準確度時：不要只報 mAP。果農在乎的是「這棵樹有幾顆」，所以該報的是計數誤差（例如人工清點 30 棵樹作為基準，報平均絕對誤差與其 95% 區間），並明確說明系統的適用條件與已知失效情境。這呼應第 14 章「指標要衡量你真正要的東西」與本章 16.7 節的透明原則。

參考資料與延伸閱讀

- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A., You Only Look Once: Unified, Real-Time Object Detection, CVPR, 2016.(YOLO 原始論文)
- Redmon, J., & Farhadi, A., YOLO9000: Better, Faster, Stronger, CVPR, 2017;YOLOv3: An Incremental Improvement, arXiv, 2018.
- Synced Review, YOLO Creator Joseph Redmon Stopped CV Research Due to Ethical Concerns, Feb. 2020.(本章叫獸開講之史料來源)
- Lin, T.-Y., et al., Microsoft COCO: Common Objects in Context, ECCV, 2014.(第 14 章亦引用)
- Ultralytics YOLO 官方文件：https://docs.ultralytics.com
- Zhang, Y., et al., ByteTrack: Multi-Object Tracking by Associating Every Detection Box, ECCV, 2022.
- 補充教材投影片下載：https://page.line.me/prof(機器人叫獸)